



LB Drive – Car Rental Booking and Administrative System

TechWrap LB – Software Development Department

Contact Us:

Mohammad Al Ayoubi – +961 70 893 180

Mahmoud Al Natour – +961 78 989 360

Ghassan Al Abbani – +961 76 090 271

1 Abstract

This project presents a web-based Rent-a-Car Booking and Administrative Management System aimed at reducing the operational inefficiencies common in mid-sized car rental companies in Lebanon, where paper workflows and spreadsheets often cause double bookings, inconsistent verification, limited financial visibility, and slow customer support. The system provides a centralized platform for multiple roles: customers, employees, administrators/managers, and an optional driver role for chauffeur rentals. Customers can browse vehicles and create bookings online or in person. Online bookings are paid exclusively by credit card via Stripe, while in-person bookings support cash or card. Every booking requires customer identity verification using a driving license and government ID to improve compliance and reduce fraud.

To improve customer interaction, an AI assistant acts as the first support layer, answering questions using real-time system context (e.g., user profile data, categories, and vehicle availability up to 45 days). When the AI cannot resolve a request, the conversation can be escalated to a live chat ticket handled by employees. Administrators manage fleet inventory, pricing and categories, driver assignments, booking and payment tracking, automated contract/receipt generation, email services, audit logs, blacklists, and reporting. The system is implemented with ASP.NET Core 8.0, Entity Framework Core, and SQL Server using a clean layered (Onion) architecture.

2 Table of Contents

1	Abstract.....	2
2	Table of Contents	3
3	Table of Figures	6
4	Introduction.....	8
4.1	Background.....	8
4.2	Objective.....	8
4.3	Scope	9
4.4	Methodology	10
4.5	Limitations	11
5	Related Research.....	12
5.1	Similar Existing Products.....	12
5.1.1	Large international rental brands (enterprise platforms)	12
5.1.2	Aggregator marketplaces (search/compare platforms)	12
5.1.3	Global rental management SaaS (B2B systems)	13
5.1.4	Regional / local vendors (Lebanon-focused systems)	13
5.2	Project's Added Value	14
6	Problem Analysis	15
6.1	Project Description	15
	Customer Account (Customer Portal)	16
	Employee Account (Operational Portal)	16
	Administrator / Manager Account (Admin Control Panel)	16
	Driver Account (Driver Portal)	17
	Communication and Email Services Hub	17
6.2	Business Requirements Overview	17
6.2.1	Functional Business Requirements	17
6.2.2	Fleet and Catalog Management	17

6.2.3	Booking and Rental Operations	18
6.2.4	Customer Management, Identity Verification and Compliance	18
6.2.5	Payment Handling (Online + In-Person).....	19
6.2.6	Driver Operations and Availability	19
6.2.7	Tracking and Location Sharing (Operational Visibility).....	19
6.2.8	Email Services Hub (Mini-Gmail + Automation)	19
6.2.9	H) Risk Controls and Enforcement.....	20
6.2.10	Ratings and Feedback	21
6.2.11	Administration, Roles, and Access Control	21
6.2.12	Reporting and Dashboards	21
6.2.13	Customer Support and AI Agent (visibility rules)	21
6.3	Non-Functional Requirements:	21
6.4	Use Cases	22
6.4.1	Actor-UseCase	22
6.4.2	Main Use Cases – Steps (flow) and Role interaction.....	26
7	Design	34
7.1	Data Modeling	34
7.1.1	Context Data Model	34
7.1.2	Key-Based Data Model (Primary Keys + Core Foreign Keys)	35
7.1.3	Fully Attributed Data Model (All Tables).....	38
7.2	Data Flow Diagrams (DFD)	47
7.2.1	Context-Level DFD.....	47
7.2.2	Level 2 DFDs (for each Level-0 process).....	50
7.3	User Interface Design	59
7.3.1	Global Layout and Navigation Structure.....	60
7.3.2	Visual Design Language (Theme, Components, Consistency)	61
7.3.3	Page-Level UI Design (Main Major Views)	61
7.3.4	UI/UX Principles Applied	81
8	Development	82
8.1	Project Structure and Implementation Approach	82
8.2	Database Schema and EF Core Integration	82
8.3	Authentication, Authorization, and Security Features	83

8.4	Customer Portal Development	83
8.5	Employee Operations Development	84
8.6	Admin Control Panel Development.....	84
8.7	Booking Lifecycle and Conflict Prevention	85
8.8	Stripe Payment Integration	85
8.9	Driver Module and Live Tracking Development	86
8.10	Customer Support (Ticket + Live Chat) Development.....	86
8.11	AI Assistant Development (Customer Only)	87
8.12	Email Services Hub Development (Mini-Gmail + Automation).....	87
8.13	UI Implementation (Layouts, Components, Identity Pages).....	88
8.14	Testing, Deployment, and Deployment Considerations	88
9	Challenges	89
9.1	External API and Partner Constraints (Flight Ticketing Removed).....	89
9.2	Payment Gateway Limitations (Production Access Not Possible)	89
9.3	Budget and Hosting Instability (Multiple Server Moves).....	89
9.4	AI Assistant Provider Selection, Cost Constraints, and Logic responses	90
9.5	Complexity Growth (Many Modules in One Platform)	90
9.6	Booking Conflicts and Data Consistency	90
9.7	Background Services Reliability (Reminders + Notification Logging)	91
9.8	Time Constraints and Team Coordination	91
10	Future Work	91
10.1.1	Payment Gateway Reintegration (Real Production Payments)	91
10.1.2	Advanced AI Assistant (Beyond API Calls)	92
10.1.3	Mobile Application Development.....	92
10.1.4	Expanded Roles and Privilege Control (Accounting, HR, Fine-Grained Permissions) 92	
10.1.5	Loyalty Program and Customer Retention Features	93
10.1.6	Data Analytics and Business Intelligence Improvements	93
10.1.7	Stronger Compliance and Document Verification Enhancements	93
11	Conclusion	93
12	Bibliography	94
	Technical documentation and standards:	94
	Software engineering and architecture:	95

Security and compliance references:	95
---	----

3 Table of Figures

Figure 1: Context-Level DFD	48
Figure 2: Level 0 DFD (Major Processes)	49
Figure 3: P1 - Authentication, MFA, Reset, Admin Reset Password	50
Figure 4: P2 Level 2 — Profile + Document Verification	50
Figure 5: P3 Level 2 — Browse, Catalog, Reference Data	51
Figure 6: P4 Level 2 — Booking and Rental Workflow	51
Figure 7: P5 Level 2 — Payments (Stripe + In-person)	52
Figure 8: P6 Level 2 — Tracking and Location Sharing (Driver + Customer live)	53
Figure 9: P7 Level 2 — Documents (Contract + Receipt + Print/Export)	54
Figure 10: P8 Level 2 — Support Tickets + Live Chat + Assignment + Close	54
Figure 11: P9 Level 2 — AI Assistant (Customer only + escalation)	55
Figure 12: P10 Level 2 — Email Hub Manual Ops (Send, Drafts, Templates, Lists, Logs, Retry, Feature Flags)	56
Figure 13: P11 Level 2 — Admin Control Panel, Reports, Dashboards, Audit, Blacklist	59
Figure 14: P12 Level 2 — Notifications and Reminders Background Service	59
Figure 15 Home Page	62
Figure 16 Home Page - Section Fleet (categories)	62
Figure 17 Footer	63
Figure 18 Aside Bar	63
Figure 19 Navbar	63
Figure 20 About us	64
Figure 21 Terms & Conditions page	64
Figure 22 Browse Cars	65
Figure 23 Make Booking	65
Figure 24 My Booking	66
Figure 25 View Details of booking (Customer side) - Parital in My bookings	66
Figure 26 Rarting - Customer side (partial in My bookings)	67
Figure 27 Booking Receipt print	67
Figure 28 Tracking page (customer side)	68
Figure 29 Create new ticket issue (Customer support)	68
Figure 30 Chat page employee side	69
Figure 31 List of tickets customer side	69
Figure 32 Chat page customer side	70
Figure 33 Customer Support Portal (employee access only)	70
Figure 34 AI Chatbot	71
Figure 35 Employee Dashboard section 1	72

Figure 36 Employee Dashboard section 2	72
Figure 37 Control - Management panel.....	73
Figure 38 Audit Logs	74
Figure 39 Email Service Hub (Management)	75
Figure 40 Email templates lists	76
Figure 41 Create template	76
Figure 42 Email logs	77
Figure 43 Email details	78
Figure 44 Driver Dashboard'	79
Figure 45 Driver Schedule.....	79
Figure 46 Account Settings	80
Figure 47Admin Dashboard	81

4 Introduction

4.1 Background

In recent years, the car rental industry has increasingly shifted toward digital platforms to meet growing customer expectations for convenience, transparency, and real-time service availability. Efficient management of vehicle fleets, reservations, customer verification, and financial transactions has become essential for rental businesses seeking to remain competitive in a technology-driven market. Despite these global trends, many small and mid-sized car rental companies in Lebanon continue to rely on manual booking processes, including paper-based records and spreadsheet tools such as Excel.

These traditional methods introduce significant operational challenges. Employees are often required to manually track vehicle availability, rental histories, customer documents, and payment records, resulting in time-consuming workflows and a high risk of human error. The absence of a centralized and automated system increases the likelihood of double bookings, inconsistent data updates when multiple employees access the same records, and inaccuracies in financial reporting. Additionally, customer identity verification and contract handling are frequently managed manually, further complicating daily operations and increasing legal and operational risk.

From a customer perspective, the lack of online services forces individuals to rely on phone calls or in-person visits to check availability, confirm pricing, or complete bookings. This not only reduces accessibility and convenience but also limits the ability of businesses to respond efficiently to customer inquiries and provide timely support. As customer demand grows for online booking, secure digital payments, instant confirmation, and responsive communication channels, the limitations of manual systems become increasingly evident.

These challenges highlight the need for an integrated digital solution capable of automating core rental operations, improving data accuracy, enhancing customer interaction, and supporting modern business requirements within the local rental market.

4.2 Objective

The primary objective of this project is to develop a comprehensive digital Rent-a-Car Management System that streamlines rental operations and eliminates the inefficiencies inherent in manual booking processes. Specifically, the project aims to achieve the following goals:

Enable customers to browse available vehicles and perform online bookings with real-time availability validation.

- Support both online and in-person booking workflows, with online payments processed exclusively through secure credit card transactions using Stripe, and in-person payments supporting cash and card methods.
- Enforce mandatory customer identity verification using driving licenses and government-issued identification to ensure legal compliance and reduce fraud.
- Provide an optional driver rental feature, allowing customers to book vehicles with assigned drivers based on availability, with live shared location on maps to plan the trip and share location if customer and employee during pickup process
- Equip employees with tools for booking validation, customer support, live chat communication, and operational management.
- Integrate an AI-powered chatbot to handle common customer inquiries using real-time system data, with escalation to live employee support when necessary.
- Provide administrators with full control over fleet management, categories, blacklisted, user roles, financial reporting, booking, payments, email services, email reminders in background, email lists and templates, automated contract and receipt generation, and system configuration.

4.3 Scope

The scope of this project focuses on the design and implementation of a web-based Rent-a-Car Management System intended for use by mid-sized rental companies. The system covers the core operational, administrative, and customer interaction processes required to manage daily rental activities efficiently.

The project **includes** the following functionalities:

- 1 Online and in-person vehicle booking management with real-time availability validation.
- 2 Online payment processing restricted to credit card transactions using Stripe, and in-person payments supporting both cash and card methods.
- 3 Mandatory customer identity verification through driving license and government-issued identification for all booking types.
- 4 Optional driver rental functionality, allowing customers to rent vehicles with assigned drivers based on availability.
- 5 GPS-based vehicle tracking or real-time driver location monitoring on Google Maps
- 6 Role-based access control supporting customers, employees, administrators, and drivers.
- 7 Employee tools for booking validation, customer verification, and customer support through live chat.
- 8 Integration of an AI-powered chatbot to handle common customer inquiries using contextual system data, with escalation to live employee support when required.
- 9 Automated generation of rental contracts and payment receipts.
- 10 Email communication services including templates, saved drafts, contact lists, sent email inside the website with no need to head to the Gmail, and automated reminders and notifications.

- 11 Manage the Email service like background reminder and link each feature with specific template and list.
- 12 Create and save and code with live output for the email templates via the live html compiler.
- 13 Audit Logs
- 14 Email Logs and Notification logs.
- 15 Multi-Factor Authentication.
- 16 Special Dashboards for each role.

The project **excludes** the following aspects:

1. Native mobile applications (the system is implemented as a web-based platform only).
2. Integration with third-party insurance providers or government databases for automatic license or ID validation.
3. Advanced analytics or machine learning models beyond rule-based and context-driven AI chatbot responses. The chatbot is using now Gemini Flash 2.0 API.
4. In email service, inside the mini-Gmail, you can not view the inbox, you can only send and save drafts.

The scope of this project is limited to delivering a functional, secure, and scalable prototype that demonstrates practical solutions to real-world operational challenges within the car rental domain, rather than a fully commercialized enterprise product.

4.4 Methodology

The project was developed using a structured and iterative software engineering methodology that emphasizes modular design, scalability, and maintainability. The development process followed successive phases of analysis, design, implementation, and testing, allowing system requirements to be refined progressively based on functional needs and technical constraints.

A **Clean Layered Architecture** (Onion) was adopted to ensure a clear separation of concerns between system components. The system is divided into four layers which are **web, application, core, and infrastructure layers**. Each Layer responsible for a specific set of tasks. This architectural approach improves code maintainability, simplifies testing, and enables independent evolution of system components.

The application was implemented as a web-based system using **ASP.NET Core 8.0 MVC** as the primary backend framework. Business logic and application workflows were implemented within the application layer, while data persistence was managed using **Entity Framework Core** with **Microsoft SQL Server**. Authentication and role-based authorization were implemented using **ASP.NET Core Identity** to securely manage users across customer, employee, administrator, and driver roles.

External service integrations were isolated within the Application layer in Services section. Online payment processing for customer bookings was implemented using the **Stripe API** to securely handle credit card transactions. Location-based features and map visualization were supported through the **Google Maps API**. Email communication functionalities, including notifications, reminders, and operational messages, were implemented through an email service **Mailjet API** integrated into the system. Customer interaction was enhanced through an AI-powered chatbot implemented using the **Gemini Studio API**, providing automated responses based on contextual system data, with escalation to live employee support when required.

The user interface was developed following the MVC design pattern, utilizing **HTML5, CSS3, JavaScript, Tailwind, and Bootstrap** to deliver responsive and consistent user experiences. This separation between UI concerns and backend logic ensures clarity and maintainability across the application.

The development process followed an agile-inspired iterative approach, beginning with requirement gathering through discussions with local rental businesses. This was followed by database modeling, system architecture design, and incremental feature development. Continuous testing and validation were performed throughout development to ensure functional correctness, security, and data consistency. The final stages focused on integration testing, deployment preparation, and documentation.

Team collaboration was organized to promote cross-functional involvement, allowing each team member to contribute to frontend development, backend logic, and database design across different system modules, while assigning responsibilities based on individual strengths.

4.5 Limitations

Although the system provides an integrated solution for car rental operations, several limitations were identified during development due to scope boundaries, external dependencies, and practical constraints.

- 1 **External verification constraints:** Customer verification is performed by employees based on uploaded driving license and government ID documents. The system does not integrate with official government or third-party services to automatically validate document authenticity.
- 2 **Payment method limitations:** Online bookings are restricted to credit card payments processed through Stripe. Cash and in-person card payments are supported only through employee-managed in-person booking workflows, which rely on manual confirmation procedures.
- 3 **AI chatbot dependency and accuracy:** The chatbot relies on an external AI provider (Gemini API). As a result, response quality depends on API availability, latency, and model behavior, and the chatbot may occasionally produce incomplete or inaccurate answers. For this reason, escalation to live employee support is required for complex cases.

4 **Email service boundaries:** The integrated “mini-Gmail” module is limited to composing, sending, and saving drafts within the system. Full inbox management (reading received emails, mailbox syncing, and email history from external providers) is outside the current implementation.

5 **Platform limitation:** The project is delivered as a web-based application and does not include native mobile applications.

These limitations define opportunities for future enhancements, including stronger external verification, extended email capabilities, improved AI governance, and broader testing and deployment hardening.

5 Related Research

5.1 Similar Existing Products

The car rental industry has seen significant digital transformation globally, with various online booking and management systems available in the market. A comprehensive review of existing solutions reveals both established international platforms and emerging regional competitors.

5.1.1 Large international rental brands (enterprise platforms)

Some international brands operating in Lebanon offer online booking and corporate services via mature web platforms (e.g., **Avis Lebanon**).

Typical features

Online reservations and fleet catalog browsing
Standardized policies and customer service processes
Corporate leasing / longer-term rentals (common for enterprises)

Strengths

Stable, proven booking experience
Strong brand trust and standardized operations

Weaknesses / gaps (relative to your FYP context)

These systems are built for **large-scale operations**, not small/mid companies transitioning from manual work.

They don’t necessarily solve a Lebanese SME’s need for **custom internal workflows** (employee verification, flexible local operational rules, internal communications tooling).

5.1.2 Aggregator marketplaces (search/compare platforms)

Platforms like **KAYAK** provide car-rental comparisons in Lebanon and list multiple rental providers and locations.

Typical features

Price comparison, search, and provider discovery
Redirected booking flow depending on supplier

Strengths

Excellent for customer discovery and comparison
Helps users find deals across suppliers

Weaknesses / gaps

Not an internal management system (limited/no back-office operations)
Doesn't address operational problems such as verification workflows, staff assignment, contract generation, auditability, or internal support processes.

5.1.3 Global rental management SaaS (B2B systems)

Dedicated rental software suites such as **Rent Centric** market full operational coverage including online reservations, GPS tracking, and financial reporting.

Typical features

Online reservation integration (including WordPress-based booking site integrations)
Fleet operations + reporting
GPS tracking support (as advertised by vendors)

Strengths

"All-in-one" operational coverage
Mature feature sets for scaling fleets

Weaknesses / gaps (for Lebanese SMEs)

Often priced and packaged for broader markets (subscription cost, complexity)
May enforce workflows that are not aligned with Lebanese practices (hybrid payment flows, local verification habits, internal comms preferences).
Customization and vendor lock-in can be limiting for small businesses.

5.1.4 Regional / local vendors (Lebanon-focused systems)

Lebanon-based vendors advertise car booking management products. Example: **Easysoft "Car Booking Management (CBM)"** promotes customer/reservation management, availability tracking, reports, and vehicle management.

Easysoft also markets separate **GPS tracking/fleet management** solutions (device-based tracking), which indicates that tracking is often treated as a distinct product line rather than being integrated into booking workflows.

Strengths

Local market presence and local support potential
Usually closer to local operational context

Weaknesses / gaps

Publicly available information often emphasizes basic booking/admin management more than advanced customer interaction and escalation support.

Many local tools appear oriented toward operations rather than a unified “customer experience + back-office + communication” platform.

5.2 Project’s Added Value

Compared to international booking portals, aggregators, and generic rental management software, the proposed system provides added value by delivering an end-to-end operational platform tailored to the workflows of a rental company, rather than focusing only on online reservations.

A) End-to-end company management and role coverage – All in One Web Application

A major contribution of the project is the unified coverage of the full operational lifecycle across all relevant actors. The platform supports **customers, employees, administrators/managers, drivers, and customer support staff**, enabling each role to access dedicated workflows and dashboards. This role coverage ensures that the system manages the company “from A to Z,” including reservations, verification, operational follow-up, reporting, and staff coordination.

B) Optional driver rental as a first-class workflow

Unlike many basic booking systems that treat chauffeur services as external or manual, the project models the driver as a formal system role. Customers can rent a vehicle **with or without a driver**, and driver assignment is integrated into the booking and operational flow, ensuring the chauffeur option is managed consistently with availability and scheduling constraints.

C) Customer support beyond static communication (AI + escalation)

Instead of limiting customer interaction to phone/email, the project introduces a two-layer customer support model:

An **AI chatbot** that answers common inquiries and clarifies policies using contextual system data (e.g., logged-in user context, vehicles, categories, Booking life cycles, and availability constraints).

A human support layer through **employee live chat** and **issue/ticket and escalation** for cases that require human verification, exceptions, or operational decisions.

This hybrid approach reduces employee workload while maintaining service quality when automation is insufficient.

D) Real-time operational visibility for drivers (location sharing / tracking)

The system enhances the driver workflow with **map-based visibility**, enabling operational coordination during pickup and delivery. When enabled, live location capabilities allow the driver/customer workflow to share current location to support accurate meeting points and reduce delays. This capability directly addresses a common operational friction point that many lightweight local systems do not cover.

E) Advanced communication automation through an integrated email module

A distinctive contribution of the project is the built-in email subsystem (“mini-Gmail”) that supports operational email workflows without requiring staff to leave the platform. It includes:

Sending and saving drafts,

Managing email templates and recipient lists, and save them into the database

Linking system events to specific templates (e.g., OTP, password reset, reminders, status updates, booking changes, notifications), covering a broad set of automated communications (over 17 event-driven email scenarios),

A template editing workflow that supports writing HTML templates and previewing rendered output through a live HTML preview mechanism. (like live html compiler)

This goes beyond simple “confirmation emails” by treating outbound communication as a managed operational module.

Auto reminders (background services) (e.g., Promocode expiry notify the employee, car not returned, ...), and notifications via emails (e.g., booking status changed, setting car inactive, backlisting, ...).

F) Risk control and compliance-oriented features

To reduce fraud and operational risk, the system includes:

Customer blacklisting and enforcement of restrictions within booking workflows,

Audit logging for accountability and traceability of sensitive actions,

Strong authentication measures including **multi-factor authentication**, which is not commonly implemented in small local rental systems despite the sensitivity of identity documents and financial operations.

G) Document automation for consistency and traceability

The platform generates standardized rental documents such as **contracts and receipts**, reducing manual paperwork, ensuring consistent formatting and terms, and improving traceability for employees and managers.

6 Problem Analysis

6.1 Project Description

The proposed solution is a web-based Rent-a-Car Online Booking and Administrative Management System that digitizes the complete operational workflow of a rental company. It replaces fragmented manual processes (paper records and spreadsheets) with a centralized platform that manages vehicle browsing and reservations, customer verification, payments, driver services, customer communication, and administrative reporting through role-based access control.

The system is organized around multiple roles, each with a dedicated interface and permissions aligned with real operational responsibilities:

Customer Account (Customer Portal)

Customers can browse available vehicles and categories, view details, and create bookings according to availability constraints. Customers can optionally request a rental **with or without a driver**, depending on service availability. The customer experience is enhanced through an **AI chatbot** that is visible only to customer accounts and is designed to answer common questions using contextual system information (e.g., Logged-in customer info, cars, categories, booking lifecycle rules, and availability constraints). For payment, customers perform **online booking using credit card only** via Stripe payment gateway, while in-person bookings are handled at the office.

Employee Account (Operational Portal)

Employees support day-to-day operations and customer service. They can review and process bookings, verify customers documents (driving license and government ID) to activate their accounts in order to allow the make bookings. Also, manage payment records related to bookings, and communicate with customers through support workflows. Employees may view reference data such as cars and categories for operational purposes, but they do not have full administrative control: **they cannot change core fleet/category attributes**, do not manage employees or roles, and do not access sensitive risk-control modules such as customer blacklisting. Employees also have limited access to the email module: they can use operational messaging capabilities when permitted, but **they do not have access to email configuration and provider settings pages**. Employees **also can-do live chats with the customers** that have submitted ticket issues as **they have access to the customer support** system.

Administrator / Manager Account (Admin Control Panel)

Administrators oversee the system configuration and the full business lifecycle through a centralized **Control Panel**. As reflected in the admin interface, the control panel provides complete management modules for core entities and sensitive operational functions, including:

Fleet and reference management (e.g., **Cars, Categories**)

Risk control (e.g., **Blacklists**)

User operations (e.g., **Customers, Employees, Change Role**)

Commercial tools (e.g., **Promo Codes**)

Financial operations (e.g., **Payment Methods, Payments**)

Operational processing (e.g., **Bookings**)

Administrators also have access to reporting pages, audit-oriented views, and system-level oversight dashboards.

Administrations **also have full access to the email settings configurations** to manage the background services (reminders and notifications) and to link each email feature with the correct email template.

Driver Account (Driver Portal)

Drivers interact with the system through a dedicated driver interface designed around assigned missions/trips and operational schedules. The driver portal supports mission execution and the operational visibility required during pickup and delivery processes. Map-based functionality can be used to support driver coordination (e.g., viewing pickup points and location-sharing flows when enabled).

Communication and Email Services Hub

A distinguishing element of the system is its integrated email subsystem, presented as an “Email Services Hub” within the platform. This module centralizes outbound business communication and supports operational workflows such as composing and sending messages, managing distribution lists, creating and maintaining standardized email templates, and monitoring sending activity through logs and notification history. It also supports automated reminders and background notifications tied to business events. Access to advanced configuration capabilities (e.g., provider settings, sender identities, and system-level email policies) is restricted to administrators to protect system integrity.

Overall, the project provides an integrated platform that connects customer booking experience, employee operational handling, administrative control, communication automation, and driver workflows within a single system. The role-based design ensures each actor sees only what is relevant to their responsibilities, improving security, accountability, and operational efficiency.

6.2 Business Requirements Overview

6.2.1 Functional Business Requirements

6.2.2 Fleet and Catalog Management

The system shall allow authorized staff to manage the vehicle inventory, including creating, updating, and deactivating cars (**Car**).

The system shall allow authorized staff to manage vehicle categories (**Category**) used for browsing and classification.

The system shall expose a customer-facing browsing experience to search/filter vehicles and view details (Browse/Car views).

Role restrictions:

Admin/Manager: full CRUD on Cars/Categories.

Employee: view Cars/Categories for operational needs but **cannot change core attributes**

Code alignment: Car, Category entities; CarManager, CategoryManager; CarController, CategoryController, BrowseController.

6.2.3 Booking and Rental Operations

The system shall allow customers to create bookings online based on real-time availability rules (**Booking**).

The system shall prevent booking conflicts (double booking) by validating availability before confirmation.

The system shall support booking lifecycle management (Confirmed, active, canceled, rejected, awaiting return, done).

The system shall allow employees to create/manage bookings for walk-in customers (in-person flow) and at returning the car to update the status.

The system shall support optional “rent with driver” during booking, linking the booking flow with driver assignment logic.

Print contract of specific booking with info of car, customer, booking, and payment.

Refer directly to view and edit the payment.

Code alignment: Booking entity; Booking Irepo and rep BookingManager; BookingController / BookingsController; booking views.

6.2.4 Customer Management, Identity Verification and Compliance

The system shall require customer verification using **Driving License** and **Government ID** before finalizing rental workflows.

The system shall allow employees to review verification documents and update the customer verification status (**Customer**).

The system shall store verification state to enforce business rules (e.g., block unverified accounts from critical actions).

Manage customer info in which employee can edit attributes.

Add/create customer account for in-person coming customers.

Code alignment: Customer entity; CustomerManager; customer admin views.
(Exact field names are in Core entity; the business requirement stays high-level here.)

6.2.5 Payment Handling (Online + In-Person)

The system shall support **online booking payments via credit card only** using Stripe integration.

The system shall support **in-person payments** (cash/card) recorded/processed by employees.

The system shall maintain payment records linked to bookings (**Payment**) and supported methods (**PaymentMethod**).

The system shall allow administrators to manage supported payment methods.

Edit on payment like amount, status, apply promocode...

Print receipt of payment.

Code alignment: Payment, PaymentMethod, CreditCard, CustomerCreditCard; PaymentManager, PaymentMethodManager; StripeController, PaymentController, PaymentMethodController.

6.2.6 Driver Operations and Availability

The system shall support a dedicated driver role with its own workflow and dashboard.

The system shall store driver availability and mission data (**DriverAvailability, Trip, TripStatus**).

The system shall allow assignment and scheduling of driver trips linked to bookings where driver service is requested.

Code alignment: Driver, DriverAvailability, Trip, TripStatus; DriverManager, TripManager.

6.2.7 Tracking and Location Sharing (Operational Visibility)

The system shall provide map-based tracking/visibility features for driver missions.

The system shall allow capturing driver location pings (**DriverLocationPing**) and expose them through tracking views/hub.

The system shall support “share live location” operationally (customer ↔ driver coordination) when enabled.

Code alignment: DriverLocationPing; TrackingController; DriverTrackingHub.

Note for your report quality: this requirement should be phrased as **operational tracking/location sharing** (as above), not “fleet GPS hardware tracking,” unless you truly implemented that.

6.2.8 Email Services Hub (Mini-Gmail + Automation)

The system shall provide an integrated Email Services Hub that supports:

6.2.8.1 *Operational mail handling*

Compose and send outbound emails from within the system (no need to open external Gmail).

Save and manage drafts (**EmailDraft**).

Maintain sent/outbox tracking and delivery logs (**EmailLog**).

6.2.8.2 *Templates and configuration*

Create and manage email templates stored in the database (**EmailTemplate**).

Support a live HTML preview/editing experience for template design.

Manage distribution lists and recipients (**DistributionList**, DistributionListMember, rules).

6.2.8.3 *Event-driven automation + background services*

Link system events/features to templates via configuration (**EmailFeatureConfig**).

Execute reminder/notification background processing and record outcomes (**NotificationProcessingService**, NotificationLog, ServiceRunItem, ServiceRunRecord).

Support multiple event-driven communications (OTP, password reset, booking updates, status changes, blacklist notifications, promo expiry, overdue return, etc.).

Role restrictions:

Admin/Manager: full access including Email Configuration Center and provider settings.

Employee: limited use (send, drafts, operational usage) but **no access to configuration/provider settings**.

Code alignment: Email-related entities listed above; EmailManager, EmailTemplateManager, EmailDraftManager, EmailLogManager, DistributionListManager, NotificationProcessingService, EmailProviderSettingsManager, SenderIdentityManager; controllers like EmailServicesController, SendEmailController, TemplatesController, DistributionListsController, EmailConfigurationCenterController.

6.2.9 H) Risk Controls and Enforcement

The system shall support customer blacklisting to restrict problematic users (**BlackList**).

The system shall enforce blacklist rules in booking and access workflows.

Role restriction: only Admin/Manager can manage blacklist entries (employee excluded).

Code alignment: BlackList; BlacklistManager; BlacklistController.

6.2.10 Ratings and Feedback

The system shall allow management and review of customer ratings (**CustomerRating**), supporting monitoring of service quality.

Customers can only do the ratings by their accounts with no option to delete it or edit on it, and the employees and admin can only view them.

Code alignment: CustomerRating; CustomerRatingManager; RatingsManagementController.

6.2.11 Administration, Roles, and Access Control

The system shall enforce role-based access control for Customer, Employee, Admin/Manager, Driver.

The system shall allow admins to manage employees and user roles (including “Change Role” module).

Employees must not access employee management or role assignment features.

Code alignment: Identity entities (AspNetUser, AspNetRole...); EmployeeManager; EmployeeController; ControlPanelController + role management pages.

6.2.12 Reporting and Dashboards

The system shall provide dashboards tailored to each role (admin, employee, driver, customer).

The system shall provide reporting capabilities for operational and financial insights.

Code alignment: DashboardController, ReportsController, and supporting views.

6.2.13 Customer Support and AI Agent (visibility rules)

The system shall provide customer support workflows enabling customer-to-employee communication and escalation.

The AI chatbot shall be available **only to customer accounts** to answer common inquiries and reduce support load.

6.3 Non-Functional Requirements:

Security: authentication and role-based authorization must protect sensitive modules (blacklist, role management, email configuration, reports).

Auditability: critical actions must be traceable through audit logs (**AuditLog**) and operational logs (email logs, notification logs).

Reliability: booking availability validation must be consistent to prevent overbooking and data conflicts.

Usability: UI must be responsive and role-specific (separate dashboards, restricted menu items).

Maintainability: layered architecture and manager-based business logic must support future changes without rewriting UI and data layers.

Integration resilience: external services (Stripe, Mailjet, Google maps, Gemini) should fail gracefully with clear error handling and logs.

6.4 Use Cases

6.4.1 Actor-UseCase

Actors

- **Customer**
 - **Employee** (Operations + Support)
 - **Administrator/Manager**
 - **Driver**
 - **System / Background Services**
 - **External Systems:** Stripe, Email Provider (Mailjet), Google Maps, AI Provider (Gemini)
-

6.4.1.1 Customer Use Cases (UC-A01)

UC-C01 Register account → account created

UC-C02 Login → authenticated session

UC-C03 Logout → session ended

UC-C04 Enable/Use Multi-Factor Authentication (MFA) → secure login

UC-C05 View/Edit profile → profile updated

UC-C06 Upload verification documents (License + Gov ID) → verification pending

UC-C07 View verification status → sees approved/rejected/pending

UC-C08 Browse cars → car list displayed

UC-C09 Filter/sort cars (category/price/features/date) → refined results

UC-C10 View car details → car info shown

UC-C11 Check availability for dates → availability result shown

UC-C12 Create booking (online flow) → booking request created

UC-C13 Select “Rent with driver” (optional) → driver requested in booking

UC-C14 Confirm booking details → booking ready for payment/processing

UC-C15 Pay online via Stripe (card only) → payment completed + booking confirmed

UC-C16 View booking list (upcoming/past) → bookings visible

UC-C17 View booking details → booking info shown

UC-C18 View contract/receipt → documents accessible

UC-C19 Rate/review rental experience → rating stored

UC-C20 Visit dashboard

UC-C21 Chat with AI assistant (customer only) → AI conversation saved

UC-C22 Ask AI about cars/categories/availability/policies → answer returned

UC-C23 Escalate from AI to human support → support conversation created

UC-C24 Live chat with employee support after submitting ticket issue → real-time support messages

UC-C25 Share live location to driver/employee (when enabled) → location shared

UC-C26 Receive notifications (email/system) → informed of changes/status

UC-C27 Reset password (Forgot password) → reset completed via OTP/email

UC-C28 Verify OTP (if used) → verification passed

6.4.1.2 Employee Use Cases (UC-A02)

UC-E01 Employee login/logout → authenticated employee session

UC-E02 View dashboard (employee-specific) → operational widgets shown

UC-E03 View cars list (read-only) → can view fleet info

UC-E04 View categories list (read-only) → can view reference data

UC-E05 Search/view customers → customer profile displayed

UC-E06 Create customer account on behalf of walk-in → customer created

UC-E07 Edit customer profile (allowed fields) → customer updated

UC-E08 Review customer documents → docs displayed

UC-E09 Approve verification → customer verified + logs/notification

UC-E10 Reject verification → customer rejected + reason + notification

UC-E11 View bookings list → bookings visible

UC-E12 Create in-person booking (walk-in) → booking created

UC-E13 View Bookings and Payments

UC-E14 Update booking status (workflow) → status changed + notifications

UC-E15 Assign driver to booking (if permitted by role rules) → driver assigned

UC-E16 Record in-person payment (cash/card) → payment recorded

UC-E17 View payment records (employee scope) → payment info visible

UC-E18 Generate/print contract & receipt → documents produced

UC-E19 Handle customer support inbox → sees incoming chats/tickets

UC-E20 Join support conversation → assigned/participating

UC-E21 Reply to customer via live chat → support continues

UC-E22 Escalate support to admin/manager/other employee → transferred/escalated

UC-E23 Close/resolve support conversation → marked resolved

UC-E24 Use Email Hub: send email (per permissions) → sent + logged

UC-E25 Use Email Hub: save draft → draft saved

UC-E26 View limited email logs/outbox (if allowed) → visibility for follow-up

UC-E27 View available email templates

UC-E28 Manage email lists

UC-E29 View Promocodes

UC-E30 View payment methods

UC-E31 View ratings/reviews → feedback visible.

6.4.1.3 Admin/Manager Use Cases (UC-A03)

Control Panel / Administration

- UC-A01 Admin login/logout**
- UC-A02 View admin dashboard** (full metrics)
- UC-A03 Manage cars (CRUD)**
- UC-A04 Activate/deactivate all entities** (instead of hard deleting)
- UC-A05 Manage categories (CRUD)**
- UC-A06 Manage bookings (view/edit/status)**
- UC-A07 Manage Payments**
- UC-A08 Manage customers (CRUD + verification oversight)**
- UC-A09 Manage employees (CRUD)**
- UC-A10 Change user roles / permissions**
- UC-A11 Manage blacklist (CRUD)**
- UC-A12 Enforce blacklist rules** (block booking/support/etc.)
- UC-A13 Manage promo codes (CRUD)**
- UC-A14 Promo code expiry rules** (ties to reminders)
- UC-A15 Manage payment methods (CRUD)**
- UC-A16 View/manage payments** (Stripe + in-person records)
- UC-A17 View ratings management**
- UC-A18 Reports**
- UC-A19 Export/print reports** (Like contract, receipt, print tables data)
- UC-A20 View audit logs** (filter by date/user/action)
- UC-A21 System monitoring / health summary** (if implemented)

Email Services Hub (Admin-level)

- UC-A30 Send email**
 - UC-A31 Save/view drafts**
 - UC-A32 Manage email templates (CRUD)**
 - UC-A33 Live HTML template preview/edit**
 - UC-A34 Manage distribution lists (CRUD + members)**
 - UC-A35 Manage sender identities**
 - UC-A36 Manage email provider settings**
 - UC-A37 Link “feature/event → template” configs** (OTP, reminders, etc.)
 - UC-A38 View email logs/outbox**
 - UC-A39 View notification logs**
 - UC-A40 Retry failed emails**
 - UC-A41 Enable/disable specific email features** (per feature flags/config)
-

6.4.1.4 Driver Use Cases (UC-A04)

UC-D01 Driver login/logout

UC-D02 View driver dashboard (missions/today schedule)

UC-D03 Go online/offline → availability status set

UC-D04 View assigned trips/missions

UC-D05 View trip details (pickup/dropoff, customer, timing)

UC-D06 Accept/confirm trip (if required)

UC-D07 Update trip status (e.g., pending → en route → arrived → completed)

UC-D08 Share live location / send location pings

UC-D09 View map / tracking screen

UC-D10 Report issue (trip problem / delay) (if implemented)

UC-D11 View trip history

6.4.1.5 Support Use Cases (UC-A05)

UC-S01 Customer opens support conversation after submitting ticket

UC-S02 System routes conversation to support inbox (list of all tickets with status)

UC-S03 Employee assigns/claims conversation

UC-S04 Live chat messaging

UC-S05 Escalate to another employee/admin

UC-S06 Resolve/close support ticket/conversation

UC-S07 Attach context (booking/customer info) to support (link ticket with existing booking/payment id or make it general issue)

UC-S08 Support analytics (counts/status) (at the header above the table)

6.4.1.6 AI Agent Use Cases (UC-A06)

UC-AI01 Customer starts AI chat session anytime

UC-AI02 AI loads contextual data (logged-in user, cars, categories, availability of booking them until 45 days, policies, general info, last 10 messages)

UC-AI03 AI answers FAQ/policy questions

UC-AI04 AI answers availability questions (within allowed constraints)

UC-AI05 AI guides lifecycle and steps of booking, payment, verification,...

UC-AI06 AI escalates to support (handoff conversation)

UC-AI07 Store AI conversation history (10 days only or until escalates)

6.4.1.7 System / Background Services Use Cases (UC-A07)

UC-BG01 Run scheduled reminders (promo expiry, overdue return, etc.)

UC-BG02 Trigger event-driven notifications (booking status change, blacklist action, etc.)

UC-BG03 Generate and send OTP emails

UC-BG04 Send forgot-password emails

UC-BG05 Log email send outcome (success/fail)
UC-BG06 Log notification events
UC-BG07 Write audit logs for sensitive actions
UC-BG08 Auto-generate contract/receipt upon booking confirmation
UC-BG09 Sync/update derived dashboard metrics (if implemented)

6.4.2 Main Use Cases – Steps (flow) and Role interaction

6.4.2.1 UC-01 Customer Account Registration

Primary Actor: Customer

Supporting Actors: System

Preconditions: Customer is not logged in.

Main Steps:

Customer opens the registration page.

Customer enters required information (name, email, password, etc.).

System validates input and checks email uniqueness.

System creates the customer account.

System confirms successful registration.

Expected Outcome (Postcondition): Customer account is created and can log in.

Alternate Flows:

A1: Email already exists → system rejects registration and shows error.

6.4.2.2 UC-02 Customer Submits Verification Documents (License + Gov ID)

Primary Actor: Customer

Supporting Actors: System

Preconditions: Customer is logged in and not verified.

Main Steps:

Customer opens the verification section in profile settings.

Customer uploads driving license and government ID images/files.

System validates file type/size and stores documents securely.

System updates customer verification status to **Pending Verification**.

System notifies customer that verification is pending (email/notification).

Expected Outcome: Documents are stored and customer is pending review.

Alternate Flows:

A1: Invalid file format/size → upload rejected with message.

6.4.2.3 UC-03 Employee Verifies Customer Documents

Primary Actor: Employee

Supporting Actors: System

Preconditions: Customer verification status is Pending; employee is authorized.

Main Steps:

Employee opens the customer list/search.
Employee opens the customer profile and views uploaded documents.
Employee reviews documents for completeness/validity.
Employee approves or rejects verification.
System updates verification status (Verified / Rejected).
System writes an audit log entry.
System sends a notification email to the customer.
Expected Outcome: Customer becomes verified (or rejected) and is notified.
Alternate Flows:
A1: Missing/unclear documents → employee rejects with reason; status becomes Rejected.

6.4.2.4 UC-04 Customer Enables Two-Factor Authentication (2FA)

Primary Actor: Customer
Supporting Actors: System, Email Service, Google Authenticator (TOTP)
Preconditions: Customer logged in.
Main Steps:

Customer opens Security Settings.
Customer should enter OTP received by email in order to be allowed to edit on those settings.
Customer chooses MFA method:
Email OTP, or
Authenticator App (TOTP), or
Backup codes (as recovery method).
Or all of above
If Email OTP:
System sends OTP to customer email.
Customer enters OTP.
If Authenticator:
System shows QR/secret key.
Customer scans in Google Authenticator.
Customer enters generated code.
System enables 2FA on the account.
System generates backup codes and displays them for download/save.
System confirms 2FA enabled and logs the security change.
Expected Outcome: Customer account has 2FA enabled with backup codes available.

Alternate Flows:
A1: OTP/code invalid or expired → system rejects and prompts retry.
A2: Customer loses authenticator → backup codes can be used for login recovery.

6.4.2.5 UC-05 Customer Browses Cars and Filters Availability

Primary Actor: Customer

Supporting Actors: System

Preconditions: Customer has access to browse page.

Main Steps:

Customer opens Browse Cars page.

Customer filters cars by:

Category,

Availability dates,

Car name (search).

Price per day range

System returns a filtered list of cars.

Customer opens a car details page.

Expected Outcome: Customer finds a car and proceeds to booking.

Alternate Flows:

A1: No cars match filters → system shows “No results” with suggestions.

6.4.2.6 UC-06 Customer Creates Booking + Stripe Payment (Online Card Only)

Primary Actor: Customer

Supporting Actors: System, Stripe

Preconditions: Customer is verified; car exists; selected dates valid.

Main Steps:

Customer selects a car and clicks “Book”.

Customer enters booking details:

Rental dates,

With driver or without driver,

If with driver: pickup location + trip plan,

Promo code (optional).

System validates:

Customer is verified,

Car availability for requested dates,

Driver availability if driver selected,

Promo code validity if applied.

System calculates total cost (including driver fee rules if applicable).

Customer confirms booking and clicks Pay.

System redirects customer to Stripe Checkout.

Customer completes card payment on Stripe.

Stripe returns success callback to system.

System creates/updates:

Booking record,

Payment record,

Booking status becomes **Confirmed**.

System sends confirmation emails/notifications.

Expected Outcome: Booking is confirmed and paid online.

Alternate Flows:

A1: Customer not verified → booking blocked; prompt verification.

A2: Availability conflict → booking blocked; ask to choose other dates.

A3: Driver not available → driver option rejected or booking continues without driver.

A4: Stripe payment failed/canceled → booking remains unpaid/not confirmed.

6.4.2.7 UC-07 Employee Handover: Start Rental and Sign Contract

Primary Actor: Employee

Supporting Actors: Customer, System

Preconditions: Booking status is Confirmed; customer arrives to pick up vehicle.

Main Steps:

Employee searches booking by ID/customer.

Employee confirms customer identity and booking details.

System generates rental contract (if not already generated).

Customer signs the contract (physical or recorded process).

Employee updates booking status to **In Progress**.

System logs the action and sends booking status notification email to customer.

Expected Outcome: Rental is active and car is handed over.

Alternate Flows:

A1: Customer fails identity check → employee cancels or holds booking; status updated.

6.4.2.8 UC-08 Return Vehicle and Close Rental

Primary Actor: Employee

Supporting Actors: System

Preconditions: Booking is In Progress.

Main Steps:

Employee opens the active/waiting to return booking.

System checks if return is late:

If late and not returned → status can be **Waiting to Return** (overdue state).

Employee updates booking status to **Completed/Returned**.

System sends closure notification email to customer.

Expected Outcome: Booking is completed and financial/receipt records are finalized.

Alternate Flows:

A1: Vehicle not returned by due time → system marks as Waiting to Return and triggers reminders.

6.4.2.9 UC-09 Customer Views Booking History, Prints Receipt, and Rates

Primary Actor: Customer

Supporting Actors: System

Preconditions: Customer logged in; has booking records.

Main Steps:

Customer opens “My Bookings”.

System displays booking history (past and upcoming).

Customer opens a booking details page.

Customer downloads/prints receipt.

Customer submits rating/review after completion.

System stores rating and updates rating dashboards/records.

Expected Outcome: Customer can review history and provide feedback.

Alternate Flows:

A1: Booking not completed → rating option disabled until completion.

6.4.2.10 UC-10 Customer Uses AI Agent (Customer Only) With Context

Primary Actor: Customer

Supporting Actors: System, AI Provider (Gemini)

Preconditions: Customer logged in; AI module enabled.

Main Steps:

Customer opens the AI assistant widget.

System loads contextual data for the session:

Logged-in customer info,

Cars, categories,

Booking constraints/availability window,

Last chat messages (if available).

Customer asks a question.

System sends prompt + context to AI provider.

AI returns a response.

System displays response and stores conversation history.

Expected Outcome: Customer receives automated guidance and conversation is saved.

Alternate Flows:

A1: AI cannot resolve → customer escalates to support ticket (UC-11).

A2: AI API unavailable → system shows fallback message and suggests support.

6.4.2.11 UC-11 Customer Support Ticket + Live Chat with Employee

Primary Actor: Customer

Supporting Actors: Employee (Support), System

Preconditions: Customer logged in.

Main Steps:

Customer opens Support and submits a new issue/ticket.

System creates a support ticket/conversation with status **Open**.

Ticket appears in the employee Support Inbox queue.

An employee opens the ticket; system assigns the ticket to that employee.

Customer and employee exchange messages through live chat.

Employee resolves the issue and marks ticket as **Resolved**.

Customer reviews outcome and can mark ticket as **Closed**.

System stores full chat history and status timeline for later viewing.

Expected Outcome: Support issue is handled with saved conversation history.

Alternate Flows:

A1: Employee cannot resolve → escalates/reassigns to admin/manager.

A2: Customer closes early → status becomes Closed; conversation remains viewable.

6.4.2.12 UC-12 Employee Operational Email Use (Limited Access)

Primary Actor: Employee

Supporting Actors: System, Email Service

Preconditions: Employee logged in; has permission for operational email features.

Main Steps:

Employee opens Email Services.

Employee creates a draft or selects a template/list (if permitted).

Employee writes email content and recipients.

Employee sends email or saves as draft.

System logs sending result in Email Logs.

Expected Outcome: Email is sent or draft is saved; action logged.

Alternate Flows:

A1: Employee tries to access Email Configuration Center → access denied.

6.4.2.13 UC-13 Admin Full Control Panel Management (CRUD + Settings)

Primary Actor: Administrator/Manager

Supporting Actors: System

Preconditions: Admin logged in.

Main Steps:

Admin opens Control Panel.

Admin manages:

Cars, Categories,

Customers,

Employees and roles,

Promo codes,

Payment methods and payments,

Bookings,

Blacklist.

Admin opens Email Services Hub and manages:

Templates, Lists, Logs,

Feature-to-template linking,

Provider settings/configuration center.

System stores changes, writes audit logs, and triggers relevant notifications.

Expected Outcome: Admin can configure and manage the entire platform end-to-end.

Alternate Flows:

A1: Invalid configuration (e.g., email provider settings) → system blocks save and shows error.

6.4.2.14 UC-14 Driver Views Schedule and Shares Location

Primary Actor: Driver

Supporting Actors: System, Maps

Preconditions: Driver logged in; trips exist/assigned.

Main Steps:

Driver opens Driver Dashboard/Schedule.

System shows assigned trips and timing.

Driver opens trip details and navigates to pickup/delivery.

Driver enables location sharing during mission.

System captures location updates and displays them on tracking maps to authorized users.

Driver updates trip status during execution.

Expected Outcome: Driver executes trips with operational location visibility and status updates.

Alternate Flows:

A1: Location permission denied → system continues without live tracking.

6.4.2.15 UC-15 Automated Notifications & Reminders (Background Services)

Primary Actor: System (Background Services)

Supporting Actors: Email Service

Preconditions: Features configured; templates linked.

Main Steps:

Scheduler triggers reminder/notification job.

System evaluates conditions (promo expiry, overdue return, status updates, OTP, etc.).

System selects correct email template and recipient list.

System sends email and records EmailLog/NotificationLog entries.

System records service run status (success/failure).

Expected Outcome: Automated communications are executed and fully logged.

Alternate Flows:

A1: Email sending fails → system logs failure and flags for retry/attention.

6.4.2.16 UC-16 Customer Blacklisting Enforcement

Primary Actor: Admin/Manager

Supporting Actors: System

Preconditions: Customer exists; admin logged in.

Main Steps:

Admin opens Blacklist module.

Admin adds customer to blacklist with reason and rules.

System enforces restrictions (e.g., block booking/support actions based on policy).

System logs action in Audit Logs and triggers notification if configured.

Expected Outcome: Blacklisted customer is restricted according to rules.

Alternate Flows:

A1: Admin removes blacklist entry → restrictions lifted and logged.

7 Design

7.1 Data Modeling

7.1.1 Context Data Model

The system is designed around a transactional core (**Bookings**) supported by identity management (**ASP.NET Identity**) and several operational subsystems: **verification, payments, driver missions and tracking, customer support, AI chat**, and an advanced **email services hub**.

7.1.1.1 Identity foundation (ASP.NET Identity)

AspNetUsers, AspNetRoles, and linking tables (AspNetUserRoles, claims, tokens...) provide authentication and role-based authorization.

Domain profiles reference Identity via aspNetUserId:

Customers.aspNetUserId → AspNetUsers.Id

Employees.aspNetUserId → AspNetUsers.Id

Drivers.aspNetUserId → AspNetUsers.Id

7.1.1.2 Core business domain

Categories classify Cars.

Cars are reserved through Bookings.

Bookings link customers to cars for a date range, optionally with a driver and pickup location.

Payments record booking payments (Stripe online card payments and in-person cash/card recorded by staff).

Promocodes can optionally discount bookings.

Payment methods link each payment with the method of paying

7.1.1.3 Operational control and governance

AuditLogs capture security-sensitive actions (who did what, when, and changes).

BlackList blocks/flags users (Identity-level) with the employee who performed the action.

7.1.1.4 Support and AI

SupportConversations and SupportMessages implement ticket-based live chat with employee assignment.

AiConversations and AiMessages persist customer-only AI chat history and escalation tracking.

7.1.1.5 Driver operations and tracking

Trips represent driver missions associated with a booking and track mission lifecycle status.

DriverLocationPings store live/periodic driver location updates per booking.

DriverAvailabilities store driver availability windows and recurring schedules.

7.1.1.6 Email Services Hub

A dedicated subsystem supports email composition, templates, lists, provider settings, logs, and background reminders:

Email composition: EmailDrafts

Templates: EmailTemplates

Distribution lists: DistributionLists, DistributionListMembers, DistributionListRules

Sending configuration: EmailProviderSettings, SenderIdentities, EmailFeatureConfigs

Observability & automation: EmailLogs, NotificationLogs, NotificationSettings, ServiceRunRecords, ServiceRunItems

7.1.2 Key-Based Data Model (Primary Keys + Core Foreign Keys)

Below are the core entities with their keys and important FK links (as implemented in SQL Server):

Customers

PK: Customers.userID

FK: Customers.aspNetUserId → AspNetUsers.Id

Employees

PK: Employees.employeeID

FK: Employees.aspNetUserId → AspNetUsers.Id

Drivers

PK: Drivers.driverID

FK: Drivers.aspNetUserId → AspNetUsers.Id

FK: Drivers.employeeID → Employees.employeeID (driver managed/created under employee/operations structure)

Categories

PK: Categories.categoryID

Cars

PK: Cars.CarID

FK: Cars.categoryID → Categories.categoryID

Bookings

PK: Bookings.BookingID

FK: Bookings.customerID → Customers.userID

FK: Bookings.carID → Cars.CarID

FK: Bookings.EmployeebookerID → Employees.employeeID (if in-person booking)

FK: Bookings.promocodeID → Promocodes.promocodeID (if applied)

Optional driver: Bookings.driverID (no FK defined here, but driver link exists in the schema logic)

Payment link appears both ways in schema (Bookings has paymentID, Payments has FK to Booking)

Payments

PK: Payments.paymentID

FK: Payments.bookingID → Bookings.BookingID

Promocodes

PK: Promocodes.promocodeID

CustomerRatings

PK: CustomerRatings.customerRatingID

FK: CustomerRatings.customerID → Customers.userID

FK: CustomerRatings.bookingID → Bookings.BookingID

Trips

PK: Trips.tripID

FK: Trips.bookingID → Bookings.BookingID

FK: Trips.driverID → Drivers.driverID (optional)

DriverLocationPings

PK: DriverLocationPings.driverLocationPingID

FK: DriverLocationPings.bookingID → Bookings.BookingID

FK: DriverLocationPings.driverID → Drivers.driverID

DriverAvailabilities

PK: DriverAvailabilities.driverAvailabilityID

FK: DriverAvailabilities.driverID → Drivers.driverID

SupportConversations

PK: SupportConversations.SupportConversationId

FK: SupportConversations.CustomerId → Customers.userId

FK: SupportConversations.BookingId → Bookings.BookingID (optional)

FK: SupportConversations.AssignedEmployeeId → Employees.employeeID (optional)

SupportMessages

PK: SupportMessages.SupportMessageId

FK: SupportMessages.SupportConversationId → SupportConversations.SupportConversationId

FK: SupportMessages.SenderUserId → AspNetUsers.Id

AI Conversations

PK: AiConversations.AiConversationId

FK: AiConversations.CustomerId → Customers.userId

AI Messages

PK: AiMessages.AiMessageId

FK: AiMessages.AiConversationId → AiConversations.AiConversationId

AuditLogs

PK: AuditLogs.Id

FK: AuditLogs.UserId → AspNetUsers.Id (optional nullable)

BlackList

PK: BlackList.blacklistID

FK: BlackList.userID → AspNetUsers.Id

FK: BlackList.employeeDoneBlacklistId → Employees.employeeId

Email Hub

EmailTemplates.Id, EmailDrafts.Id, DistributionLists.Id

DistributionListMembers.DistributionListId → DistributionLists.Id

DistributionListRules.DistributionListId → DistributionLists.Id

EmailDrafts.CreatedByUserId → AspNetUsers.Id

EmailLogs.CreatedByUserId → AspNetUsers.Id

EmailTemplates.UpdatedByUserId → AspNetUsers.Id

EmailProviderSettings.UpdatedByUserId → AspNetUsers.Id

SenderIdentities.CreatedByUserId/UpdatedByUserId → AspNetUsers.Id

EmailFeatureConfigs.SenderIdentityId → SenderIdentities.Id

Automation: ServiceRunItems.ServiceRunRecordId → ServiceRunRecords.Id

Notification controls: NotificationSettings.UpdatedByUserId → AspNetUsers.Id

7.1.3 Fully Attributed Data Model (All Tables)

The database schema is designed to support the complete operational lifecycle of the platform: authentication and role management, fleet and booking workflows, payments, customer verification, driver scheduling and tracking, customer support, AI chat history, enterprise-grade email services, and system auditing.

The tables are grouped below by function for clarity.

A) Platform Infrastructure (EF / Migrations)

These tables are automatically maintained by Entity Framework migrations:

__EFMigrationsHistory

MigrationId, ProductVersion

__ApplicationHistory

MigrationId, ProductVersion

B) Authentication and Authorization (ASP.NET Identity)

Identity provides secure authentication, password management, MFA flags, claims, roles, and tokens.

AspNetUsers

Id, UserName, NormalizedUserName

Email, NormalizedEmail, EmailConfirmed

PasswordHash, SecurityStamp, ConcurrencyStamp

PhoneNumber, PhoneNumberConfirmed

TwoFactorEnabled, LockoutEnd, LockoutEnabled, AccessFailedCount

AspNetRoles

Id, Name, NormalizedName, ConcurrencyStamp

AspNetUserRoles

UserId, RoleId

AspNetUserClaims

Id, UserId, ClaimType, ClaimValue

AspNetRoleClaims

Id, RoleId, ClaimType, ClaimValue

AspNetUserLogins

LoginProvider, ProviderKey, ProviderDisplayName, UserId

AspNetUserTokens

UserId, LoginProvider, Name, Value

C) Business Profiles (Role Extensions of Identity)

Identity users are extended into business role profiles.

Customers

userID, name, aspNetUserId

Verification: isVerified

Documents: drivingLicenseFront, drivingLicenseBack, nationalIDFront, nationalIDBack

Status: isactive

Address

Employees

employeeID, name, aspNetUserId

salary, address, isActive

Drivers

driverID, aspNetUserId, employeeID

driverCode, fullName, phone, Email

Status: isActive

createdAt, updatedAt

Optional quality/compliance: rating, licenseNumber, licenseExpiry, languages, notes

D) Fleet and Catalog

Categories

categoryID, name

IsActive

image

Cars

CarID, plateNumber, modelName, modelYear, color

pricePerDay, isAvailable

categoryID

carImage

E) Booking and Rental Workflow (Core Operational Flow)

Bookings is the central workflow table connecting customer, car, payment, employee involvement, promo code usage, and optional driver logistics.

Bookings

Relations: BookingID, customerID, carID, EmployeebookerID, promocodeID, paymentID

Channel: isBookedByEmployee

Dates: startdate, enddate

Pricing: subtotal, totalPrice

Status: bookingStatus

Driver option: hasDriver, driverID, driverDailyFee

Pickup: pickupLatitude, pickupLongitude, pickupLocationLabel, pickupAddress, pickupDateTime

Reminder tracking: lastPaymentReminderSentAt, lastPickupReminderSentAt,
lastReturnReminderSentAt

F) Payments and Pricing

Payments

paymentID, bookingID, amount, paymentDate

creditcardID (nullable)

paymentMethod (cash/card/creditcard naming), status

Provider integration:

paymentProvider (e.g., Stripe)

paymentProviderPaymentIntentId

paymentProviderSessionId

PaymentMethods

id, paymentMethodName, isActive

Promocodes

promocodeID, discountPercentage, validUntil

Description, name

Status: isActive, isExpiredNotificationSent

G) Customer Feedback

CustomerRatings

customerRatingID, customerID, bookingID

stars (1..5), feedback

ratingDate

H) Risk Control and Enforcement

BlackList

blacklistID, userID

reason, dateBlocked

employeeDoneBlacklistId

I) Driver Scheduling and Live Tracking

DriverAvailabilities

driverAvailabilityID, driverID

Range fields: startDateTime, endDateTime

Day/time fields: date, startTime, endTime

Scheduling flags: isRecurringWeekly, isAvailable

createdAt, updatedAt

Trips (driver mission lifecycle linked to a booking)

tripID, bookingID, driverID, tripStatus

Timeline: startedAt, arrivedAt, tripStartedAt, completedAt, cancelledAt

cancelReason

Last known snapshot: lastDriverLatitude, lastDriverLongitude, lastLocationUpdatedAt

createdAt, updatedAt

DriverLocationPings (high-frequency telemetry per booking)

driverLocationPingID, bookingID, driverID

latitude, longitude

Optional telemetry: speed, heading, accuracyMeters, batteryPercent

createdAt

J) Customer Support (Ticketed Live Chat)

SupportConversations

SupportConversationId, CustomerId, optional BookingId

Subject, Category

Status (Open, In Progress, Resolved, Closed)

AssignedEmployeeId

Timestamps: CreatedAt, UpdatedAt, optional ClosedAt

RequiresHumanIntervention

SupportMessages

SupportMessageId, SupportConversationId

SenderId, SenderRole

MessageText, optional AttachmentUrl

CreatedAt

IsInternalNote

K) AI Chat (Customer-only memory + escalation flag)

AiConversations

AiConversationId, CustomerId

CreatedAt, LastActiveAt

IsEscalated

AiMessages

AiMessageId, AiConversationId

Sender, Content

CreatedAt

L) Audit Logging (Traceability and Accountability)

AuditLogs

Actor: ActorName, ActorRole, optional UserId

Action targeting: Action, Entity, EntityId, TargetType, TargetId

Metadata: Timestamp, IpAddress, Device, UserAgent, CorrelationId

Change tracking: OldValuesJson, NewValuesJson, DetailsJson, Summary

Result fields: Status, Outcome, FailureReason

M) Email Services Hub (Mini-Gmail + Automation + Governance)

This subsystem supports operational email sending, drafts, templates, lists/rules, provider configuration, delivery logs, and background reminder processing.

EmailTemplates

Id, TemplateKey, Name, Category

Subject, Body

IsActive

UpdatedAt, UpdatedByUserId

EmailDrafts

Id, Subject, Body

RecipientsRaw, SelectedDistributionListIdsRaw

CreatedAt, UpdatedAt

CreatedByUserId

DistributionLists

Id, Name, Description, IsActive

CreatedAt, UpdatedAt

CreatedByUserId, UpdatedByUserId

DistributionListMembers

Id, DistributionListId

Email, Label, MemberType

IsActive

AddedAt, AddedByUserId

DistributionListRules

Id, DistributionListId

Rule flags: IncludeEmployees, IncludeAdmins, IncludeCustomers

Constraints: OnlyActiveUsers, ExcludeBlacklistedCustomers, OnlyVerifiedEmails

ManualEmailsRaw

UpdatedAt

EmailFeatureConfigs (event → template mapping)

Id, FeatureKey, FeatureName, Category

Enabled

SenderIdentityId, TemplateKey, ReplyToOverride

Notes

UpdatedAt, UpdatedByUserId

EmailProviderSettings (provider policy)

Id, ProviderType

SenderDomain, DefaultReplyToEmail

SandboxModeEnabled

RateLimitPerMinute

Retry policy: RetryCount, RetryDelayMinutes

UpdatedAt, UpdatedByUserId

SenderIdentities

Id, DisplayName, FromEmail, optional ReplyToEmail

IsActive, IsDefault, VerifiedStatus

CreatedAt, UpdatedAt

CreatedByUserId, UpdatedByUserId

EmailLogs (delivery logs)

Id, EmailType, TemplateKey, Subject, Body

RecipientsRaw

Status, Attempts, LastError

CreatedAt, optional SentAt

CreatedByUserId

NotificationLogs

Id, EventType

TargetType, TargetId

RecipientEmail

Result, Details

CreatedAt, Actor

NotificationSettings (automation control)

Flags: ReminderProcessingEnabled, ReminderProcessingPaused, AllNotificationEmailsEnabled

Scheduling: CheckIntervalMinutes

Payment reminders: PaymentReminderEnabled, PaymentReminderInitialDelayHours,
PaymentReminderIntervalHours, PaymentReminderMaxDurationHours,
PaymentReminderStatusCsv

Pickup reminders: PickupReminderEnabled, PickupReminderScheduleHoursCsv

Return reminders: ReturnReminderEnabled, ReturnReminderScheduleHoursCsv

Promo expiry: PromoExpiryEnabled, PromoExpiryDaysBefore, PromoExpiryAutoDeactivate,
PromoExpiryCheckFrequency

Run telemetry: LastRunAt, NextRunAt, LastRunFailedCount, LastRunProcessedCount,
LastRunSentCount, LastRunSummary

UpdatedAt, UpdatedByUserId

ServiceRunRecords (batch runs)

Id, RunAt, TriggeredBy

ProcessedCount, SentCount, FailedCount

Summary

ServiceRunItems (per-item results inside a run)

Id, ServiceRunRecordId

EventType, TargetType, TargetId

RecipientEmail

Result, optional Error

CreatedAt

7.2 Data Flow Diagrams (DFD)

7.2.1 Context-Level DFD

At the highest level, the system interacts with the following external entities:

Customer: browsing, booking, verification upload, AI chat, support tickets

Employee: verification approval, in-person bookings, booking status updates, support handling

Admin/Manager: full CRUD, blacklist, audit logs, email configuration, reports

Driver: trip schedule, status updates, location sharing

Stripe: online payment processing (card only)

Email Provider: outbound email sending (OTP, booking updates, reminders, notifications)

Google Maps: pickup location selection + live tracking visualization

Gemini AI: AI responses using system context

Outputs include: booking confirmations, receipts/contracts, status updates, automated notifications, support conversations, audit and email logs.

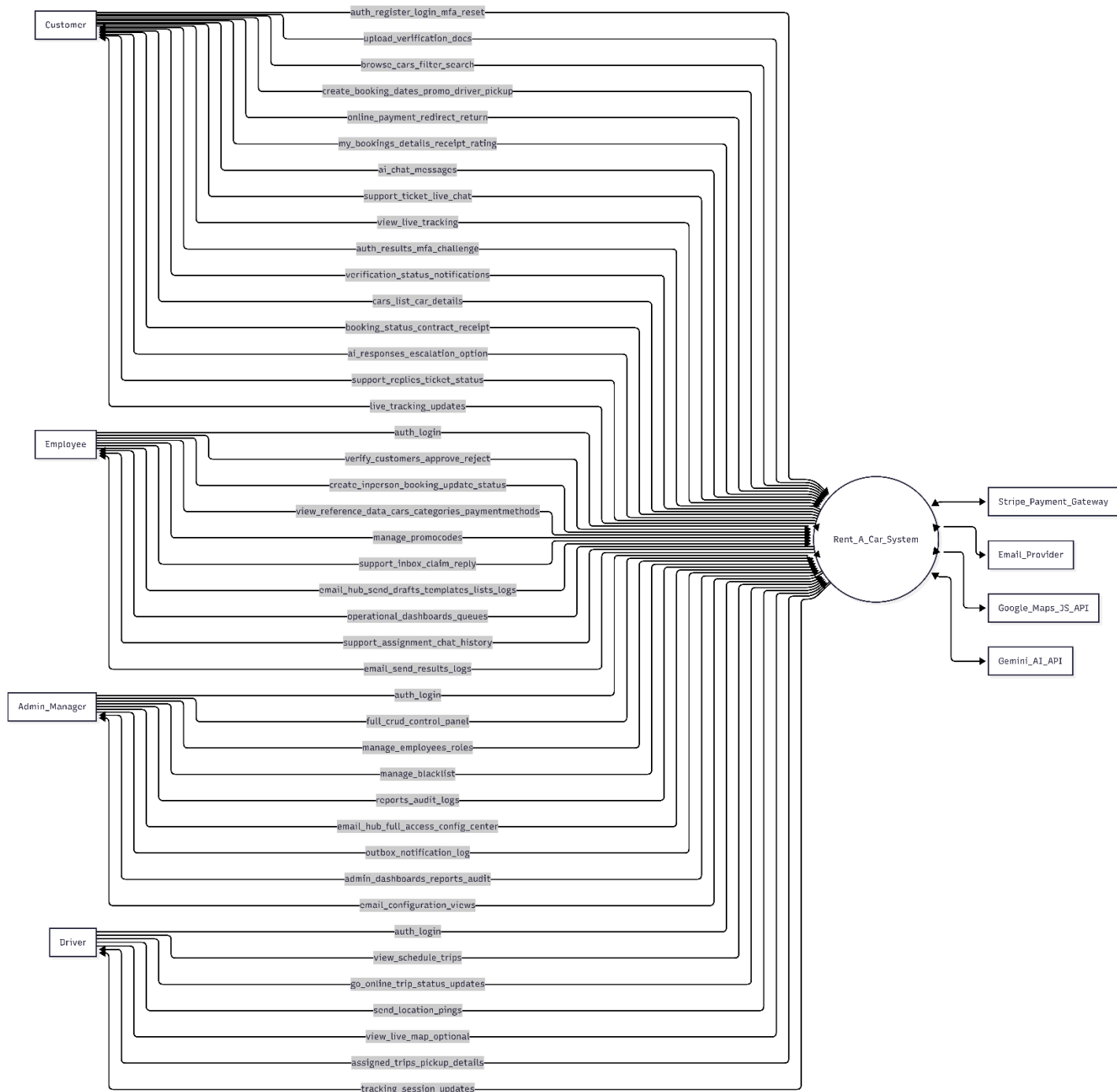


Figure 1: Context-Level DFD

7.2.1.1 Level 0 DFD (Major Processes)

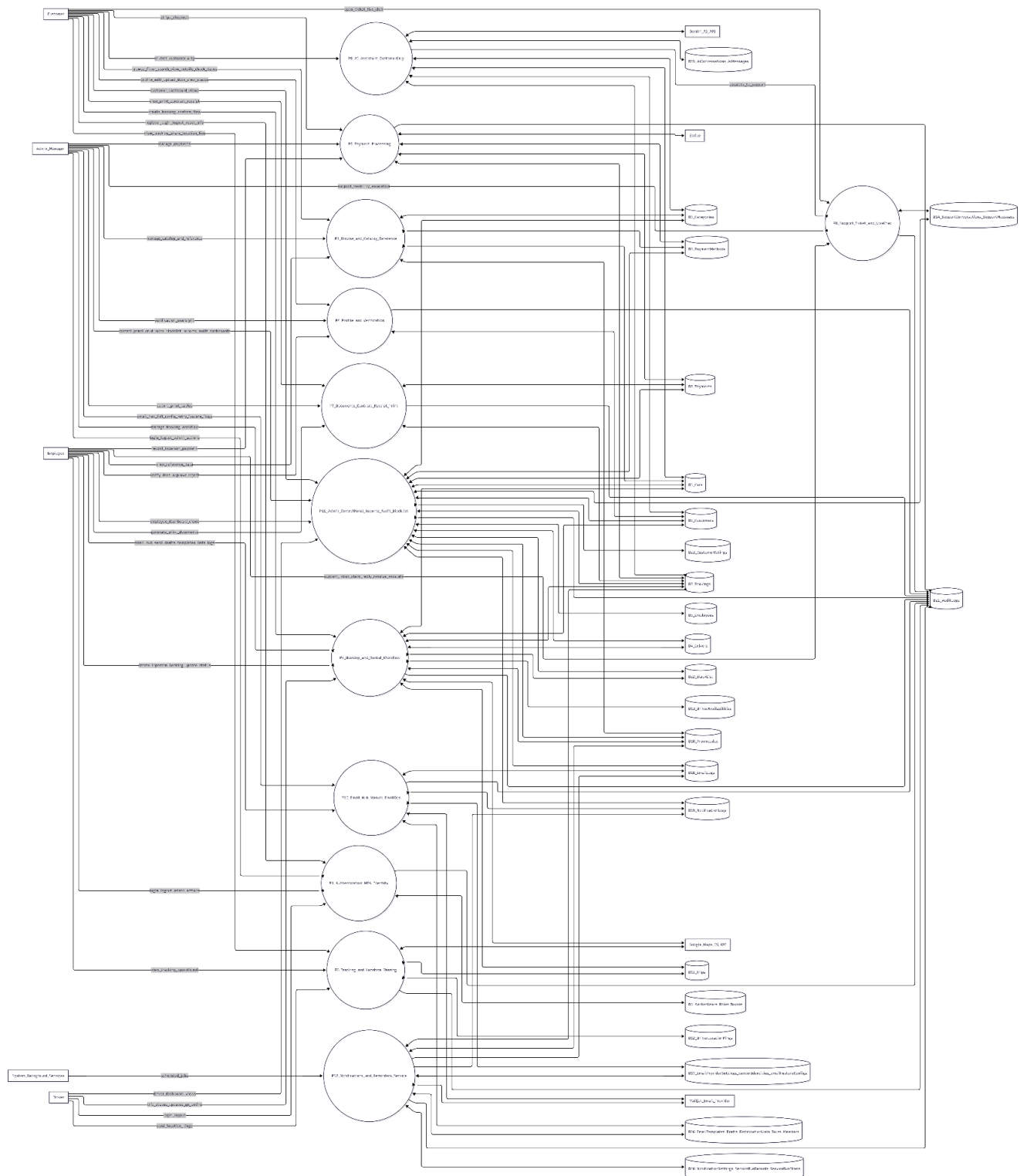


Figure 2: Level 0 DFD (Major Processes)

7.2.2 Level 2 DFDs (for each Level-0 process)

7.2.2.1 P1 - Authentication, MFA, Reset, Admin Reset Password

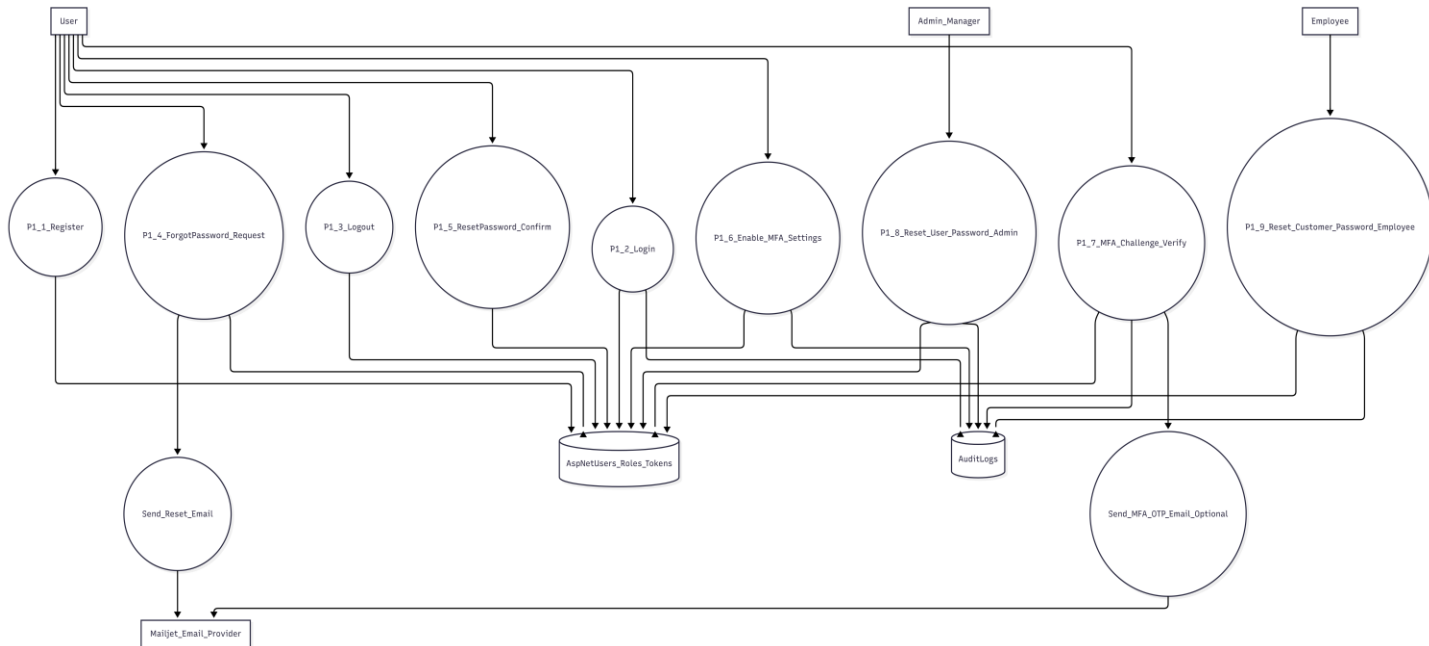


Figure 3: P1 - Authentication, MFA, Reset, Admin Reset Password

7.2.2.2 P2 Level 2 — Profile + Document Verification

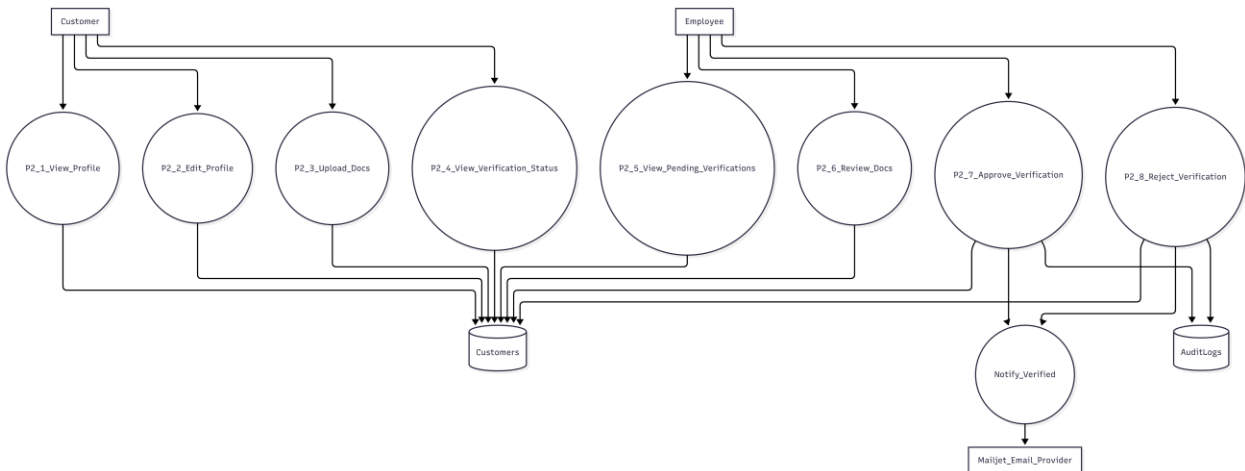


Figure 4: P2 Level 2 — Profile + Document Verification

7.2.2.3 P3 Level 2 — Browse, Catalog, Reference Data

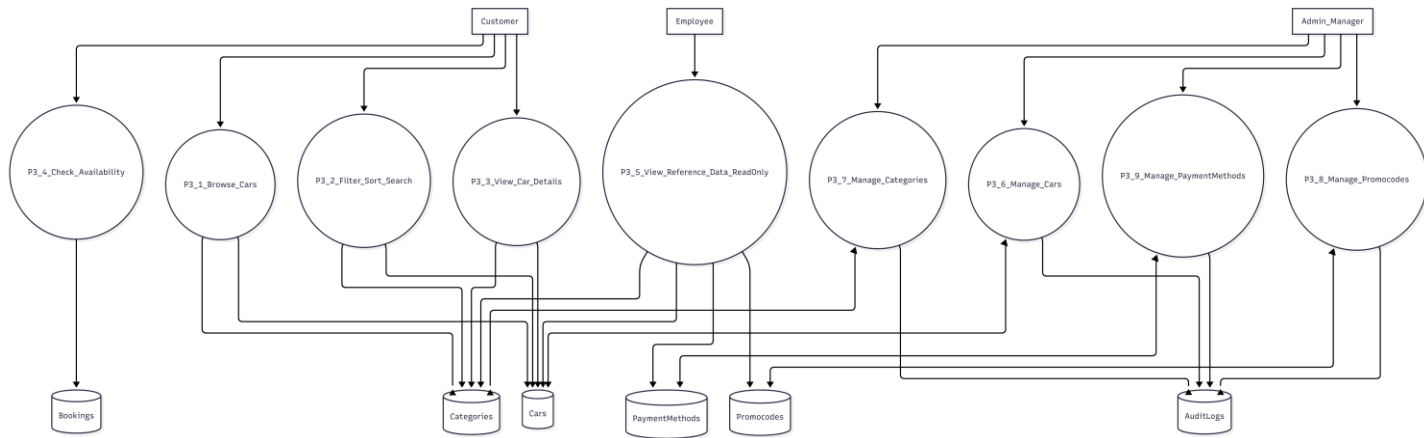


Figure 5: P3 Level 2 — Browse, Catalog, Reference Data

7.2.2.4 P4 Level 2 — Booking and Rental Workflow

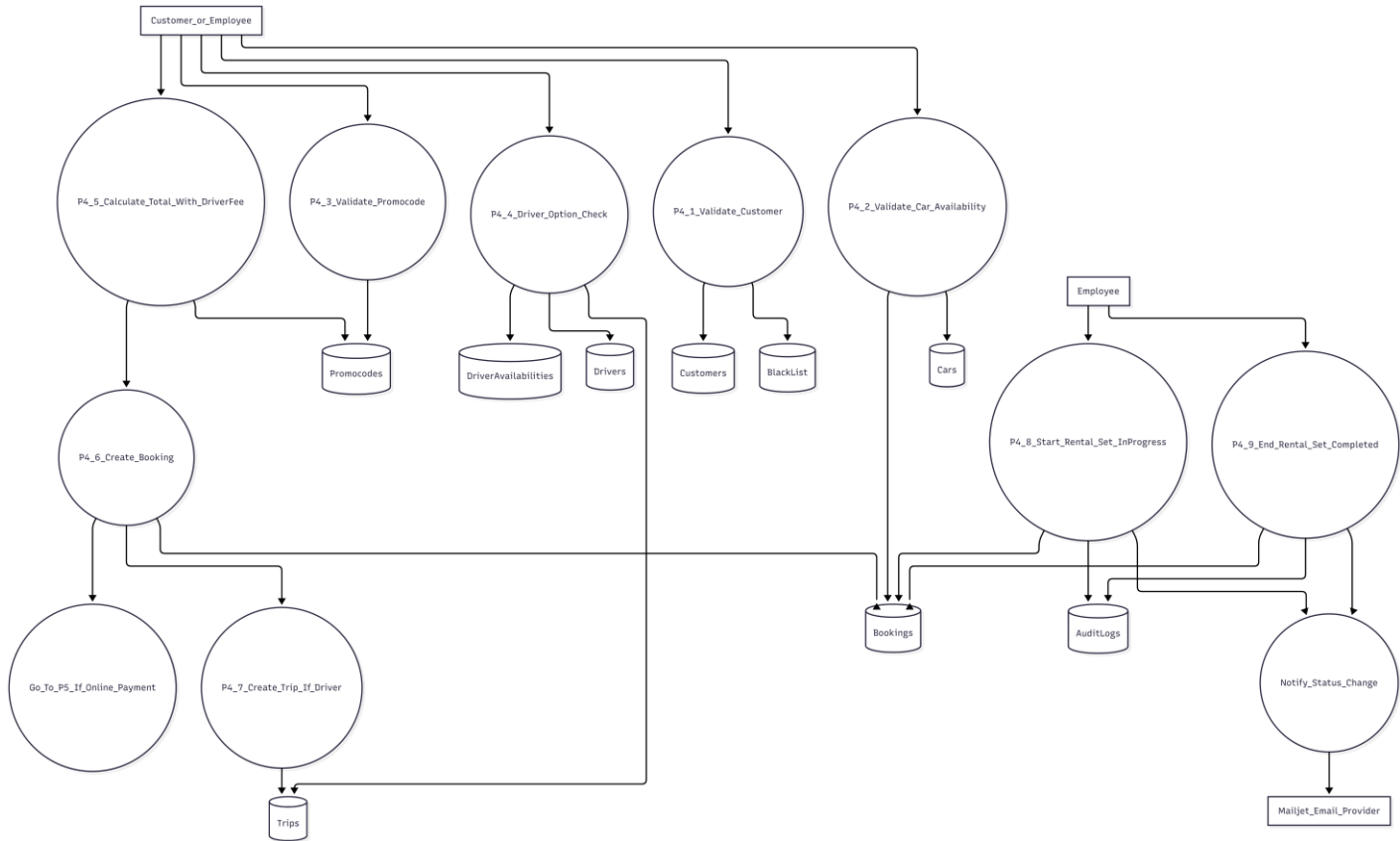


Figure 6: P4 Level 2 — Booking and Rental Workflow

7.2.2.5 P5 Level 2 — Payments (Stripe + In-person)

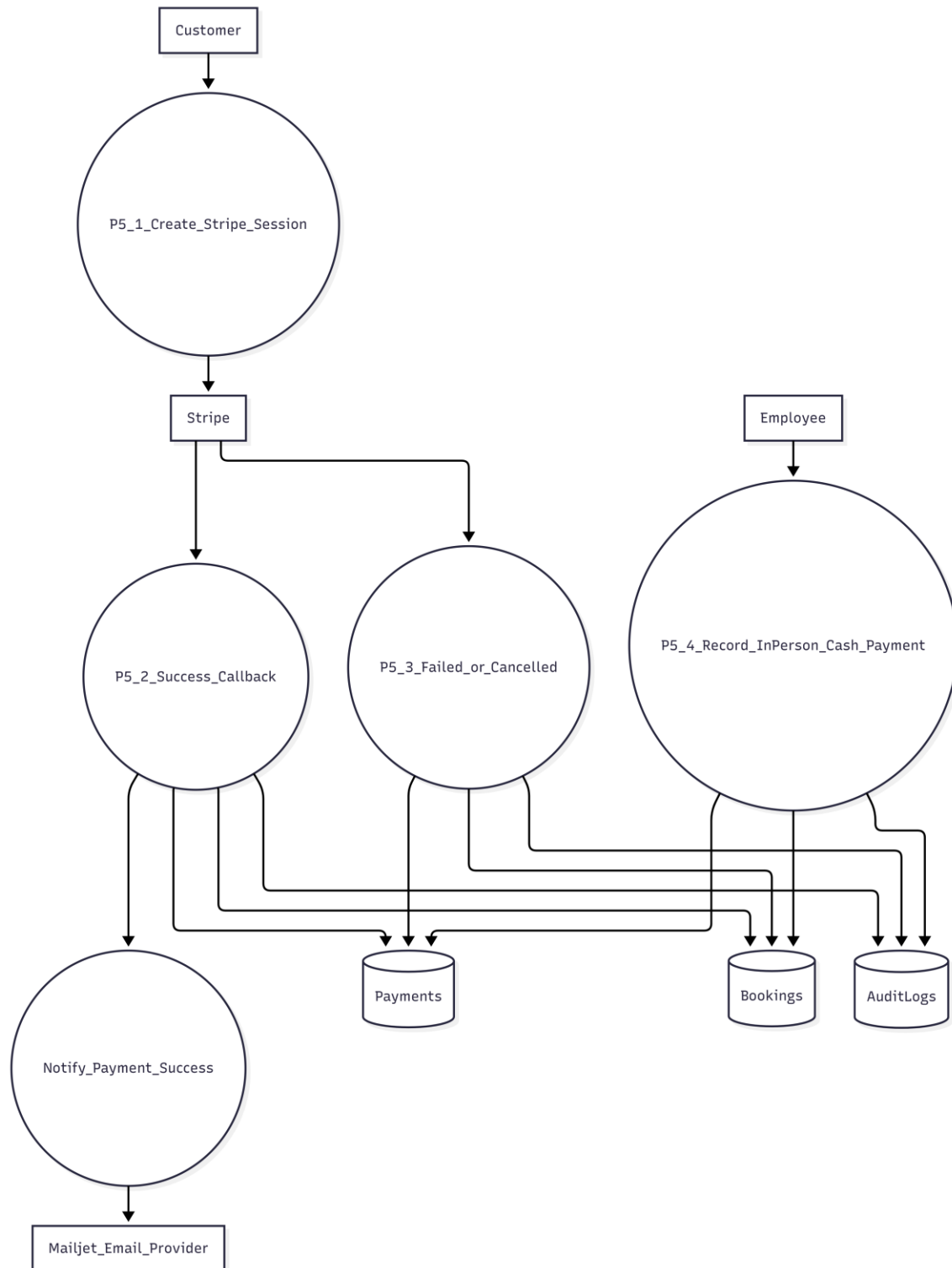


Figure 7: P5 Level 2 — Payments (Stripe + In-person)

7.2.2.6 P6 Level 2 — Tracking and Location Sharing (Driver + Customer live)

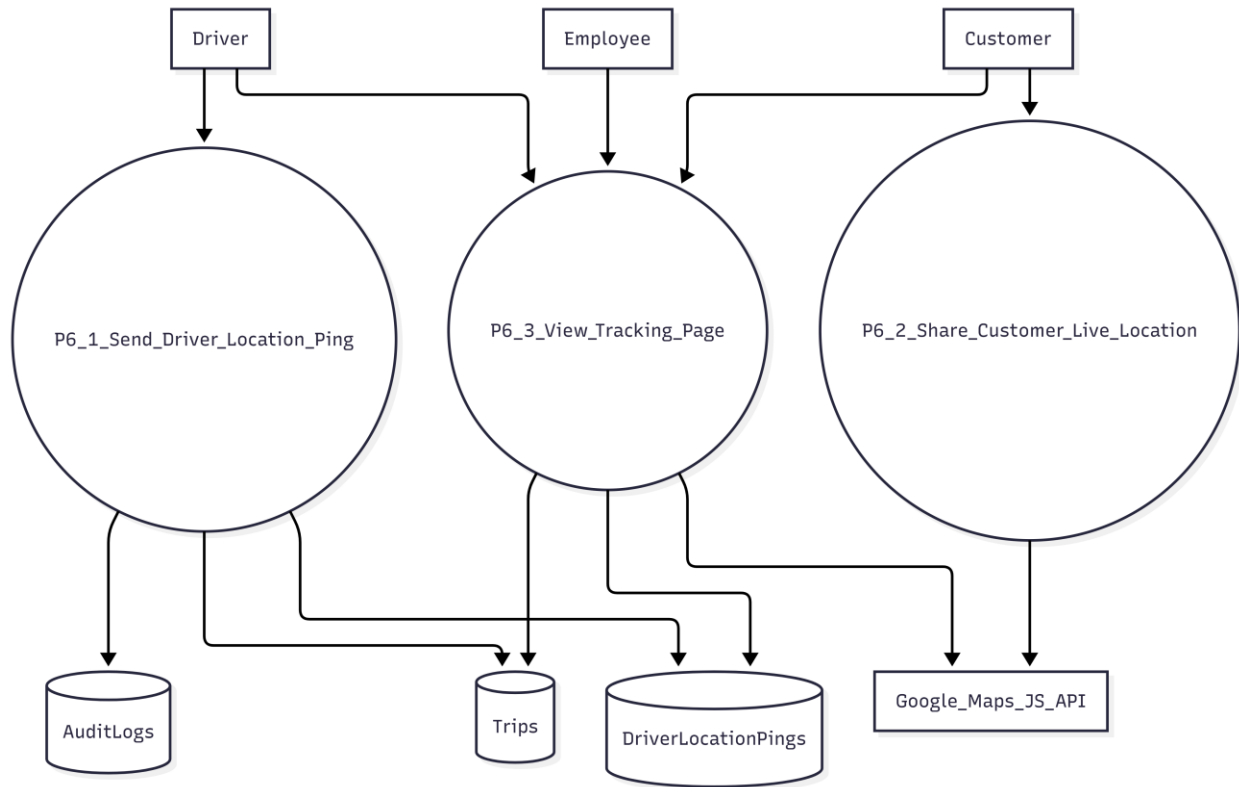


Figure 8: P6 Level 2 — Tracking and Location Sharing (Driver + Customer live)

7.2.2.7 P7 Level 2 — Documents (Contract + Receipt + Print/Export)

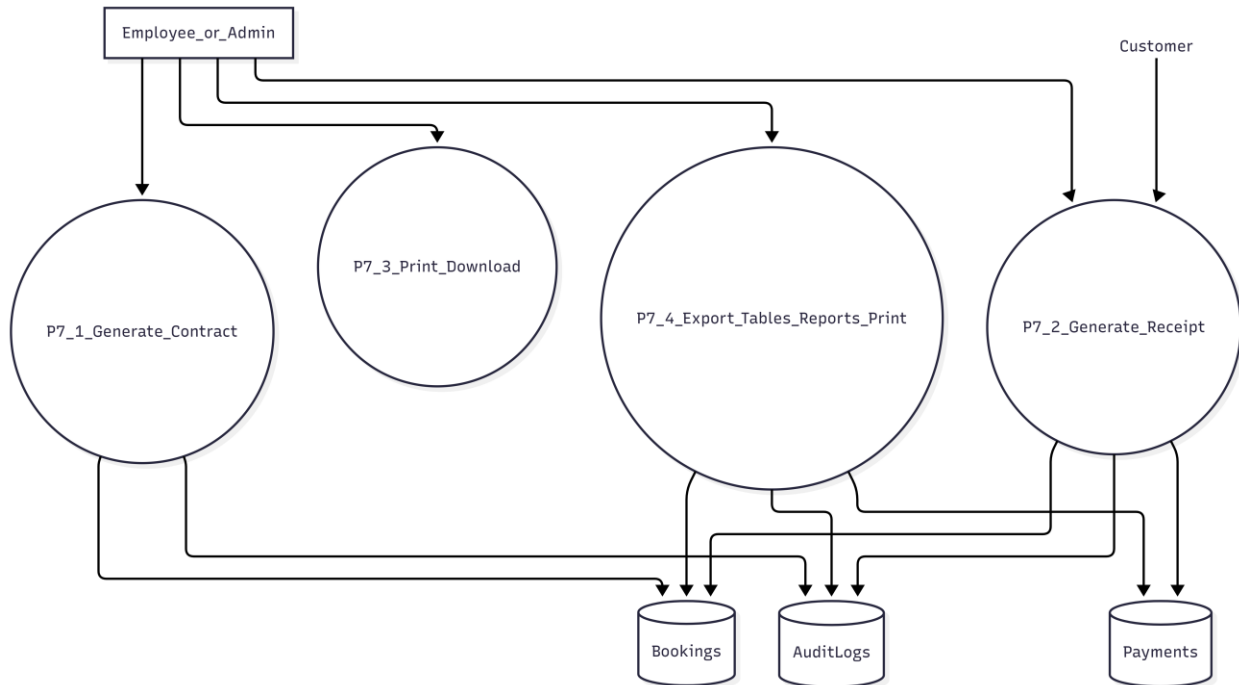


Figure 9: P7 Level 2 — Documents (Contract + Receipt + Print/Export)

7.2.2.8 P8 Level 2 — Support Tickets + Live Chat + Assignment + Close

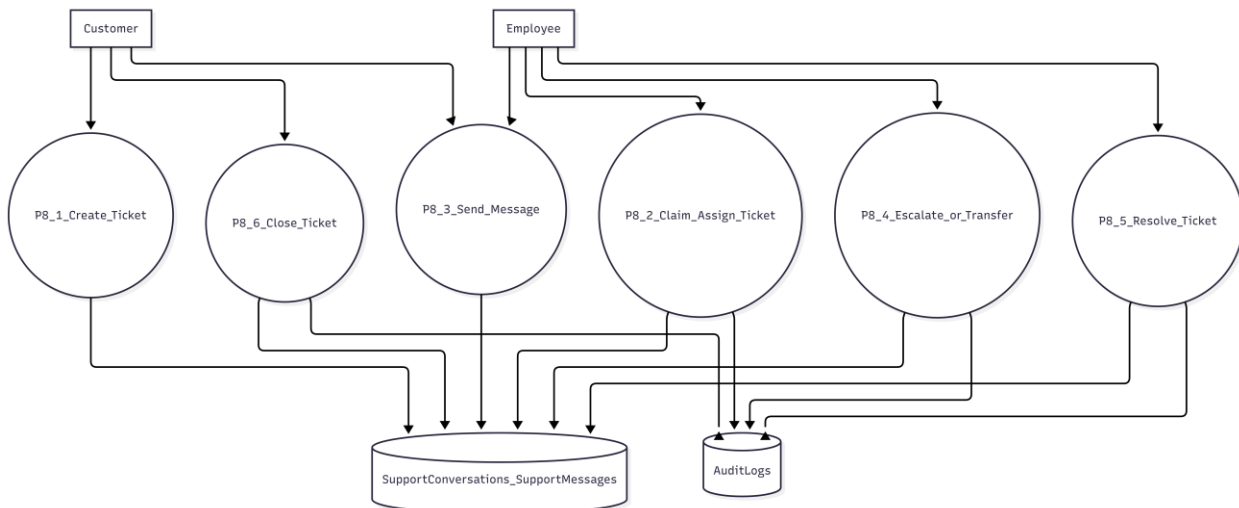


Figure 10: P8 Level 2 — Support Tickets + Live Chat + Assignment + Close

7.2.2.9 P9 Level 2 — AI Assistant (Customer only + escalation)

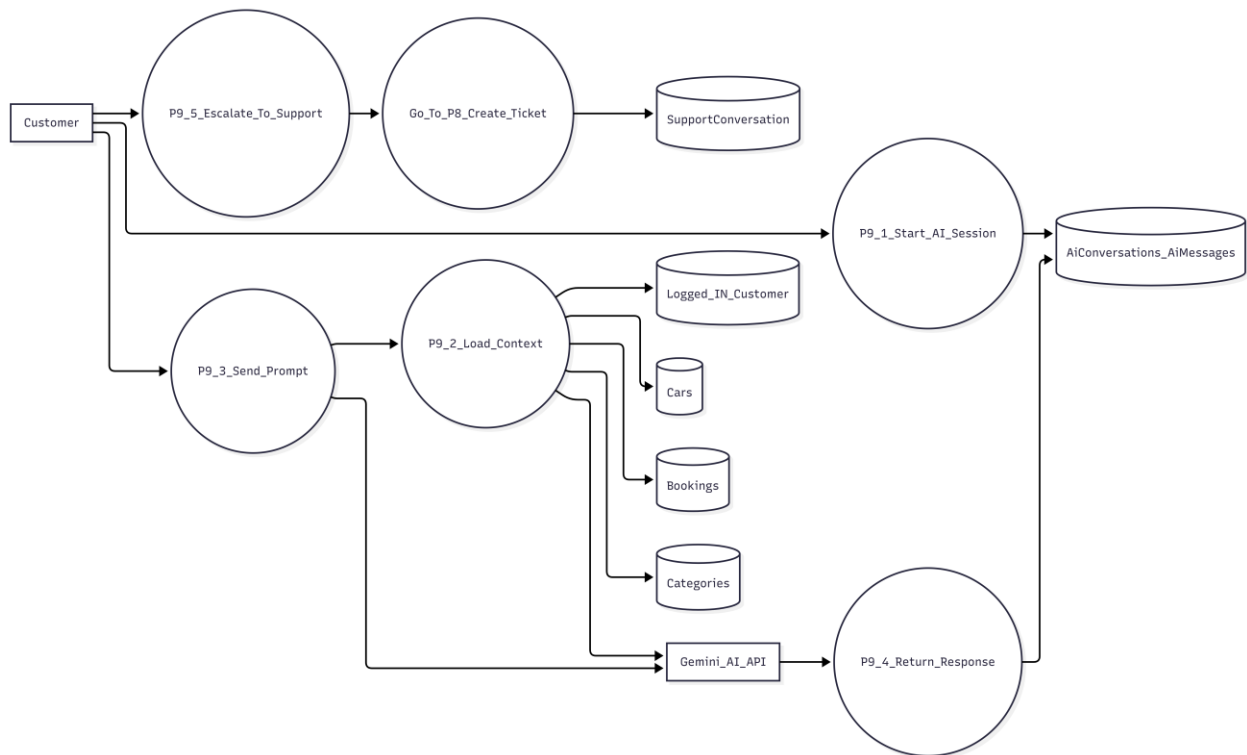


Figure 11: P9 Level 2 — AI Assistant (Customer only + escalation)

7.2.2.10 P10 Level 2 — Email Hub Manual Ops (Send, Drafts, Templates, Lists, Logs, Retry, Feature Flags)

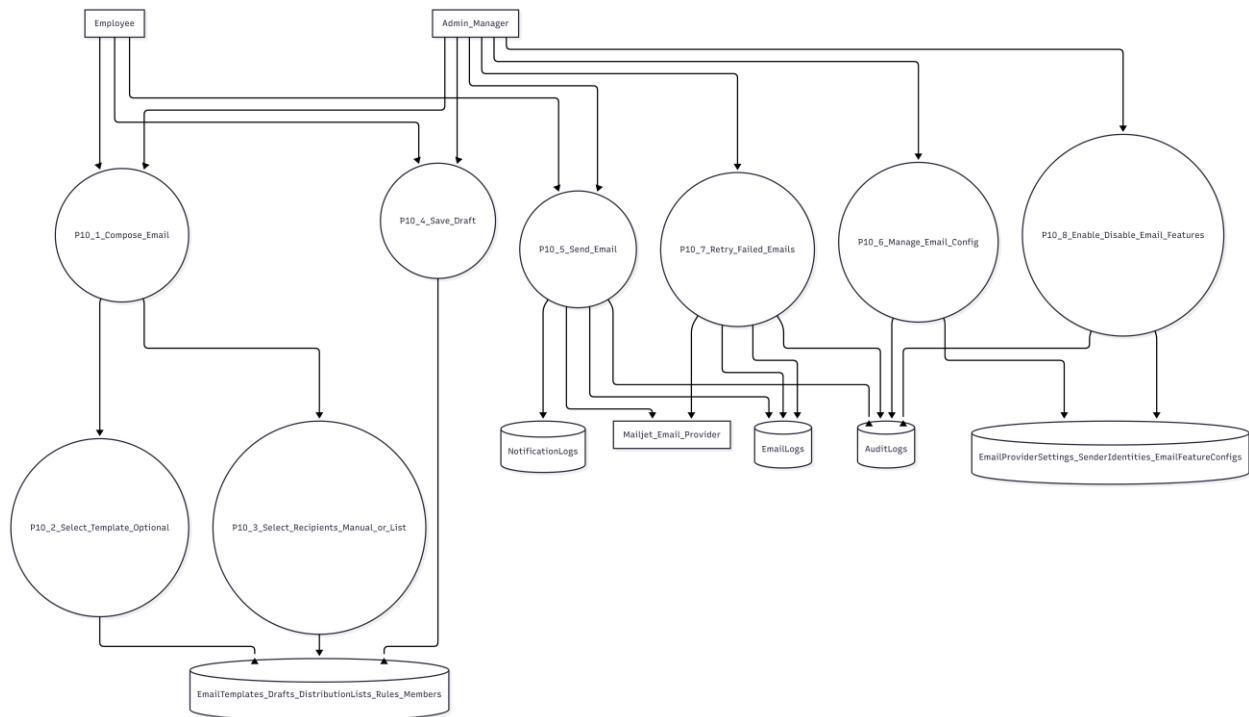
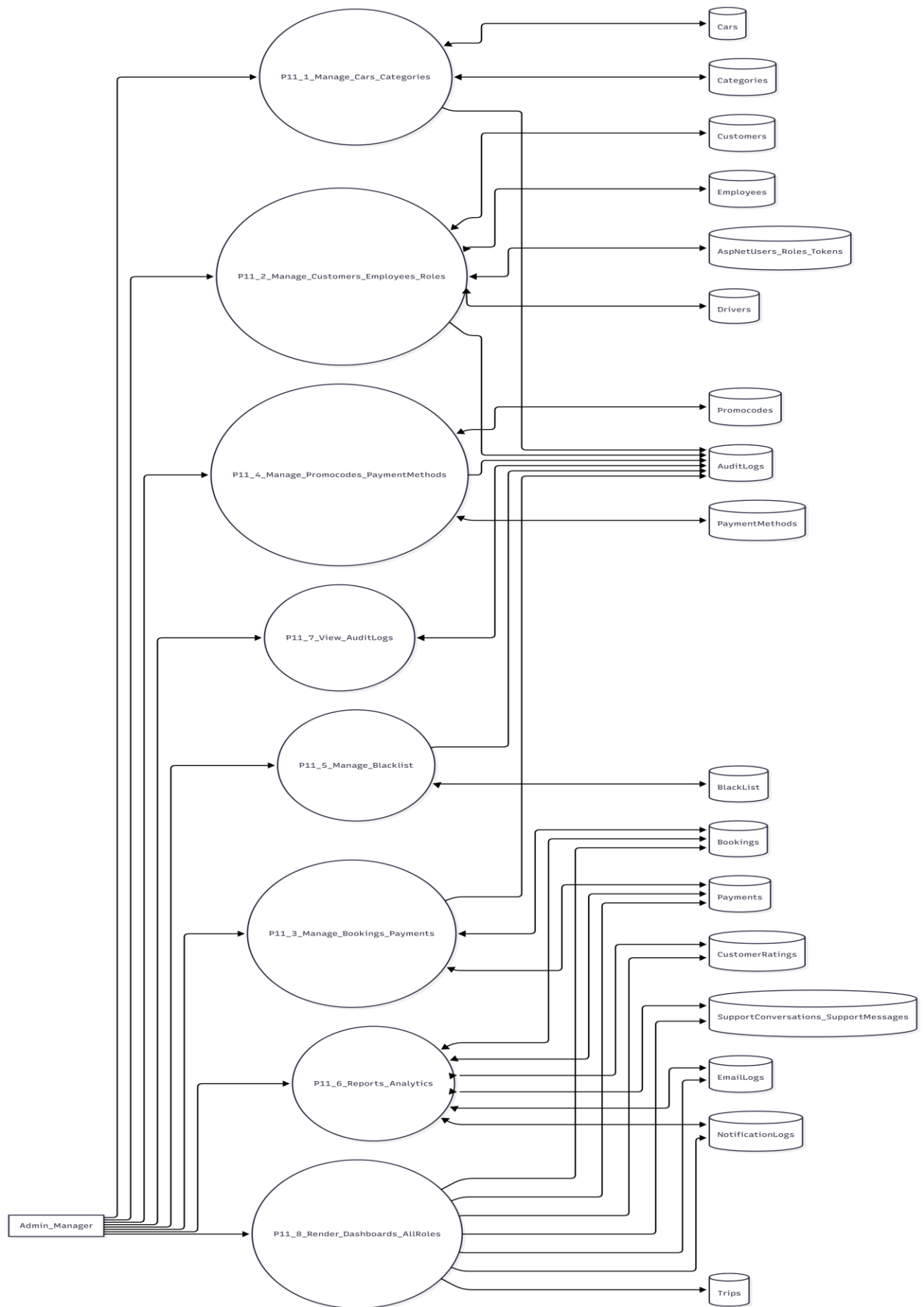


Figure 12: P10 Level 2 — Email Hub Manual Ops (Send, Drafts, Templates, Lists, Logs, Retry, Feature Flags)

7.2.2.11 P11 Level 2 — Admin Control Panel, Reports, Dashboards



7.2.2.12 , Audit, Blacklist

Figure 13: P11 Level 2 — Admin Control Panel, Reports, Dashboards, Audit, Blacklist

7.2.2.13 P12 Level 2 — Notifications and Reminders Background Service

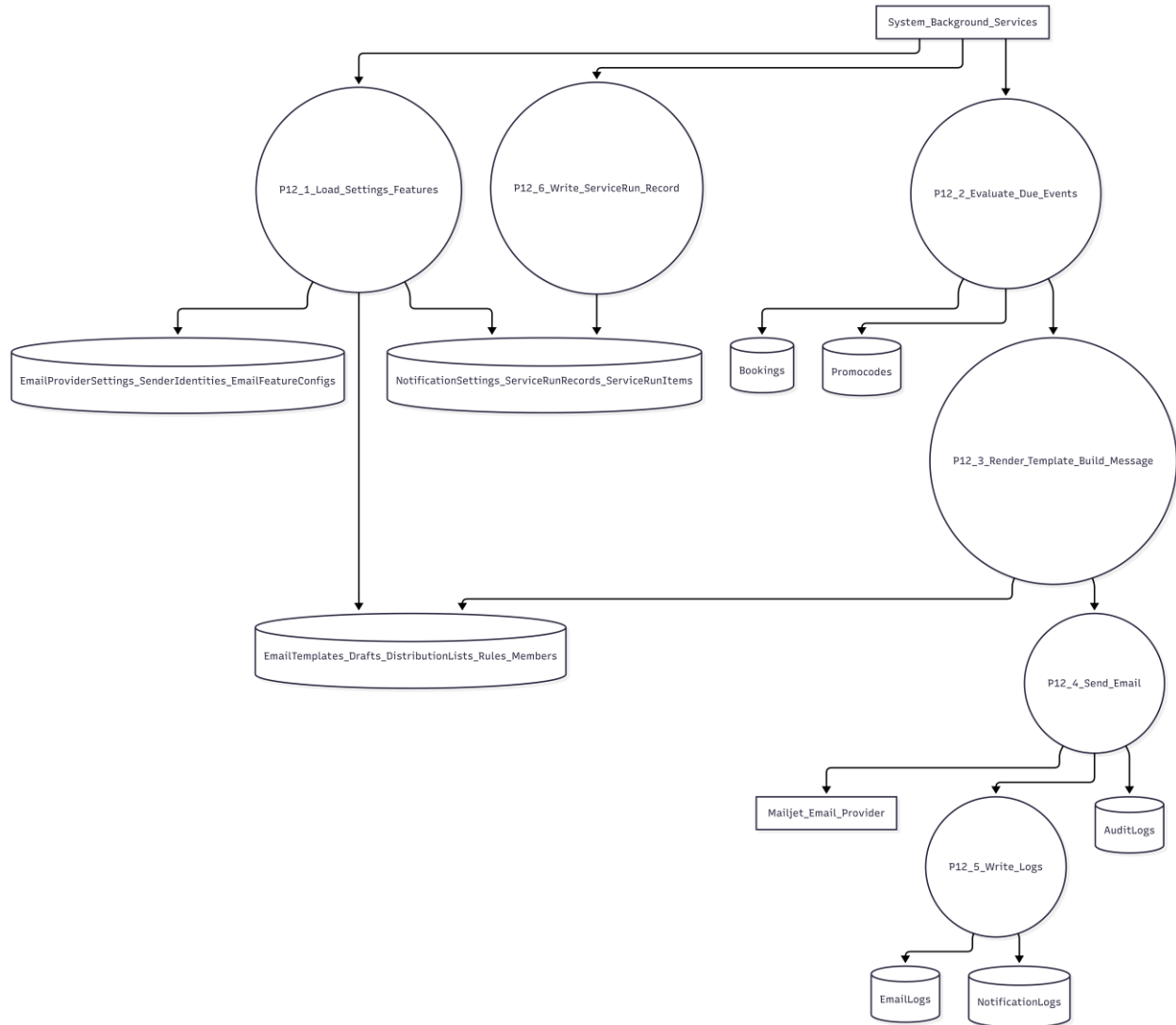


Figure 14: P12 Level 2 — Notifications and Reminders Background Service

7.3 User Interface Design

The system uses a **consistent dark-gold UI theme** across all portals (Customer, Employee, Admin, Driver) with a **gold accent color** for branding and primary actions (e.g., buttons, active menu states, key metrics). The UI is implemented using a mix of **Bootstrap (layout + components)** and **Tailwind CSS (fine-grained styling and modern visuals)**, with Tailwind configured as important: true to reliably override Bootstrap on the newer dark pages. Typography relies on modern sans fonts

(Manrope / Plus Jakarta Sans), and iconography is unified using **Google Material Symbols** (plus Font Awesome in some profile elements).

7.3.1 Global Layout and Navigation Structure

7.3.1.1 Main Layout Composition

All pages share a single layout structure:

Top Navbar (_Navbar.cshtml) for global actions and quick navigation.

Collapsible Sidebar (_Sidebar.cshtml) for role-based navigation and module grouping.

Main Content Area (RenderBody()).

Footer (_Footer.cshtml) displayed across the site, including brand, links, contact info, and subscription form.

This layout ensures that users always know:

“Where am I?” (active page highlighting in sidebar/navbar),

“What can I do next?” (modules grouped by role and function),

“How do I return?” (consistent navigation controls and breadcrumb-like grouping).

7.3.1.2 Sidebar (Role-Based Navigation)

The sidebar is a **slide-in panel** (mobile-first) with:

Brand header (“LebanonDrive”) + current role label (Customer/Employee/Admin/Driver).

Grouped sections: **Main, Management, Account**.

Hover-expand submenus for complex modules (e.g., Control Panel, Email Services).

Role-driven visibility examples:

Customer: Browse Cars, My Bookings, Help & Support, profile settings, AI chatbot through floating button.

Admin/Employee: Management - Control Panel, Email Services Hub, Reports, Support Inbox (employee access is restricted by authorization even when some links exist).

Driver: Driver Dashboard, Schedule.

AI chat widget is injected only for customers (so the UI stays clean for staff/driver accounts).

7.3.1.3 Top Navbar (Quick Access + Status Awareness)

The navbar provides:

Sidebar toggle button (responsive).

Quick links (center area) that change by role.

Notifications dropdown: used for important account states (notably identity verification reminders).

User profile dropdown: Profile + Logout.

A key UX decision here is **status visibility**: for example, if a customer is not verified, the notification bell shows an attention badge and provides a direct “Verify Now” action.

7.3.1.4 Footer (Brand + Trust + Utility)

The footer is designed like a landing-page footer:

Strong branding and short company description.

Social links.

Contact and locations.

Information links (About, Terms, Privacy).

Subscription form posting to an API endpoint with inline success/error feedback (toast/popup style).

7.3.2 Visual Design Language (Theme, Components, Consistency)

Theme and Colors

Dark surfaces for background and cards, reducing eye strain and matching modern admin dashboards.

Gold accent used for:

Primary actions (e.g., “Manage”, “Launch Module”),

Active navigation states,

Highlights and small “glow” shadows around important UI states.

Secondary text uses muted grays for hierarchy (titles white, details gray).

Core UI Components

Across most pages, the UI reuses the same building blocks:

Cards for modules and dashboard blocks (as shown in the Admin Control Panel and Email Services Hub screenshots).

Badges / status chips for booking states, verification, payment states, and ticket statuses.

Modals for create/edit/view flows (CRUD confirmation modals, detail modals for logs, support actions).

Popups/toasts/partials for feedback (_PopupNotifications.cshtml, plus specialized popups like rating/support action).

7.3.3 Page-Level UI Design (Main Major Views)

Note: Not all Views are listed and the screenshots are on some pages as there is a lot of views, but we select the most important and the major views

Below is how the UI is organized per feature area, based on the project’s Views/Controllers structure.

7.3.3.1 Public / General Pages (Home)

7.3.3.1.1 Views/Home/

Home Index acts as the marketing entry point.

Static pages: **About**, **Terms**, **Privacy** use consistent typography, spacing, and dark theme styling.

B) Customer Portal

Nav, Footer. aside

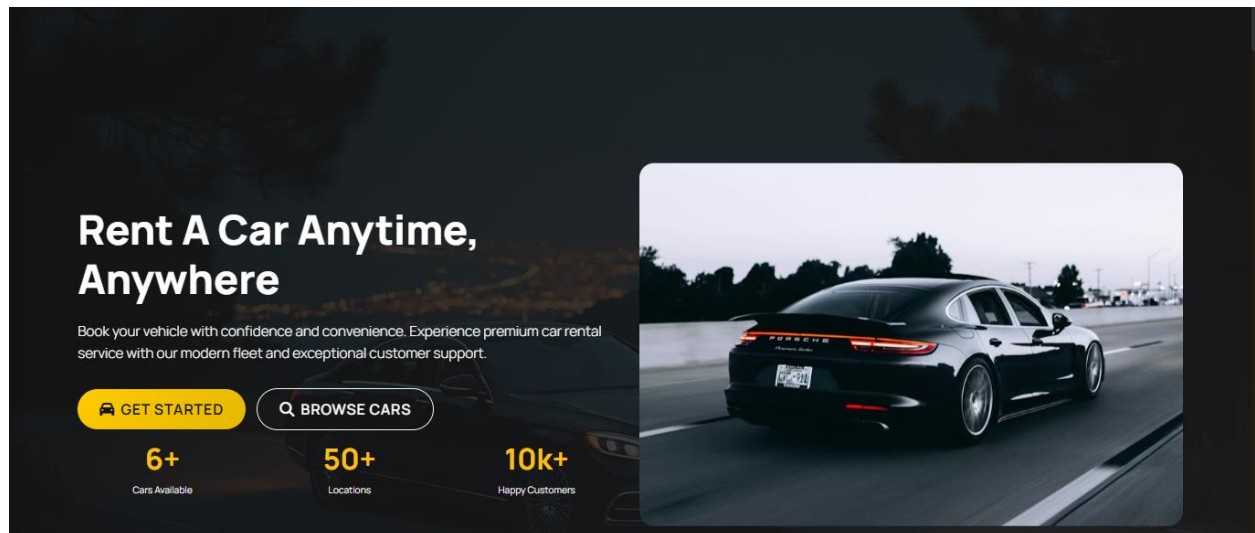


Figure 15 Home Page

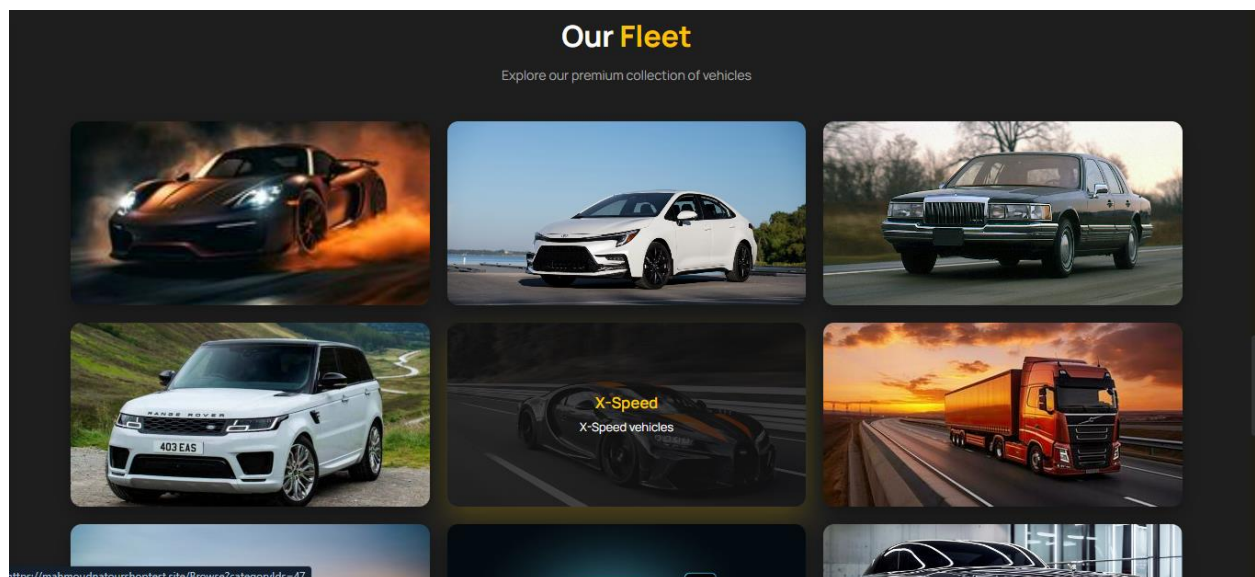


Figure 16 Home Page - Section Fleet (categories)

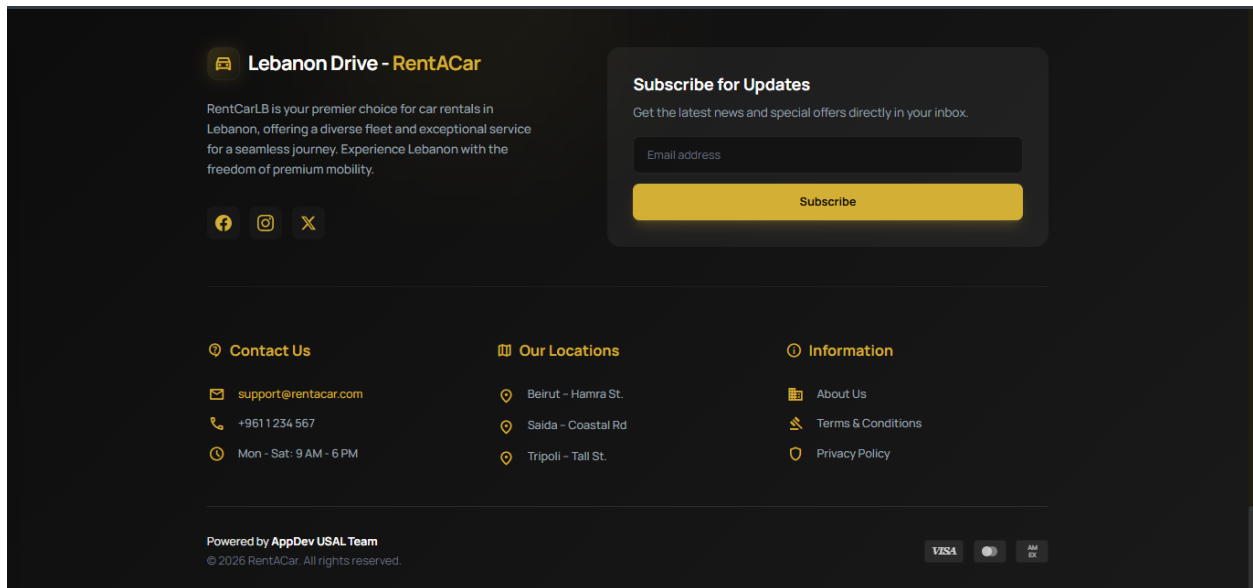


Figure 17 Footer

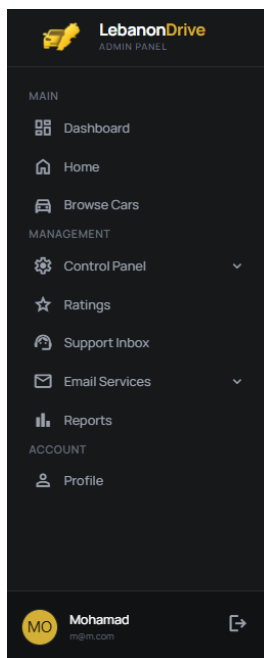


Figure 18 Aside Bar

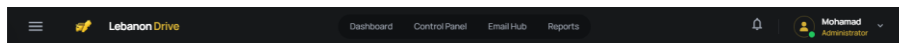


Figure 19 Navbar

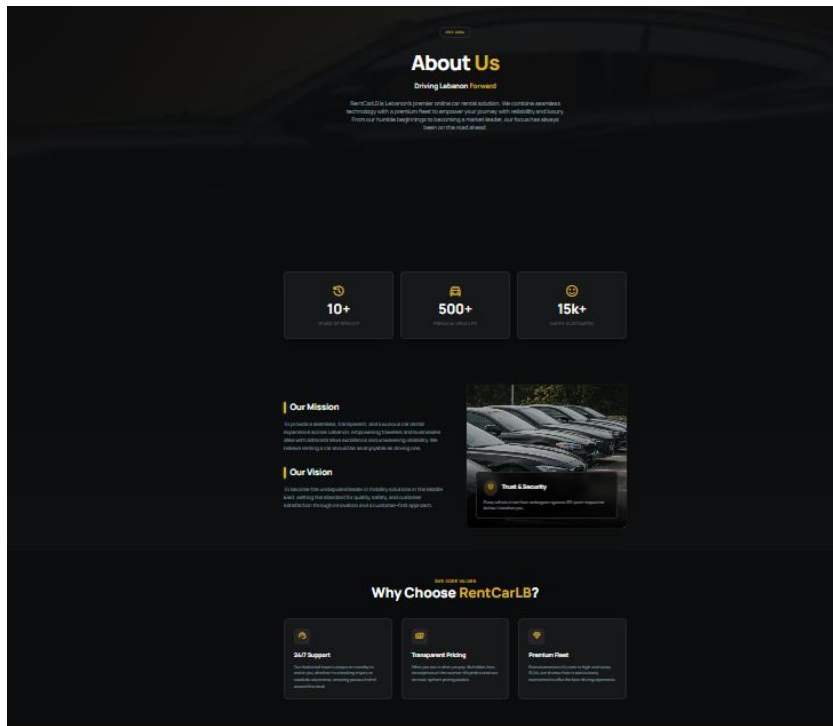


Figure 20 About us

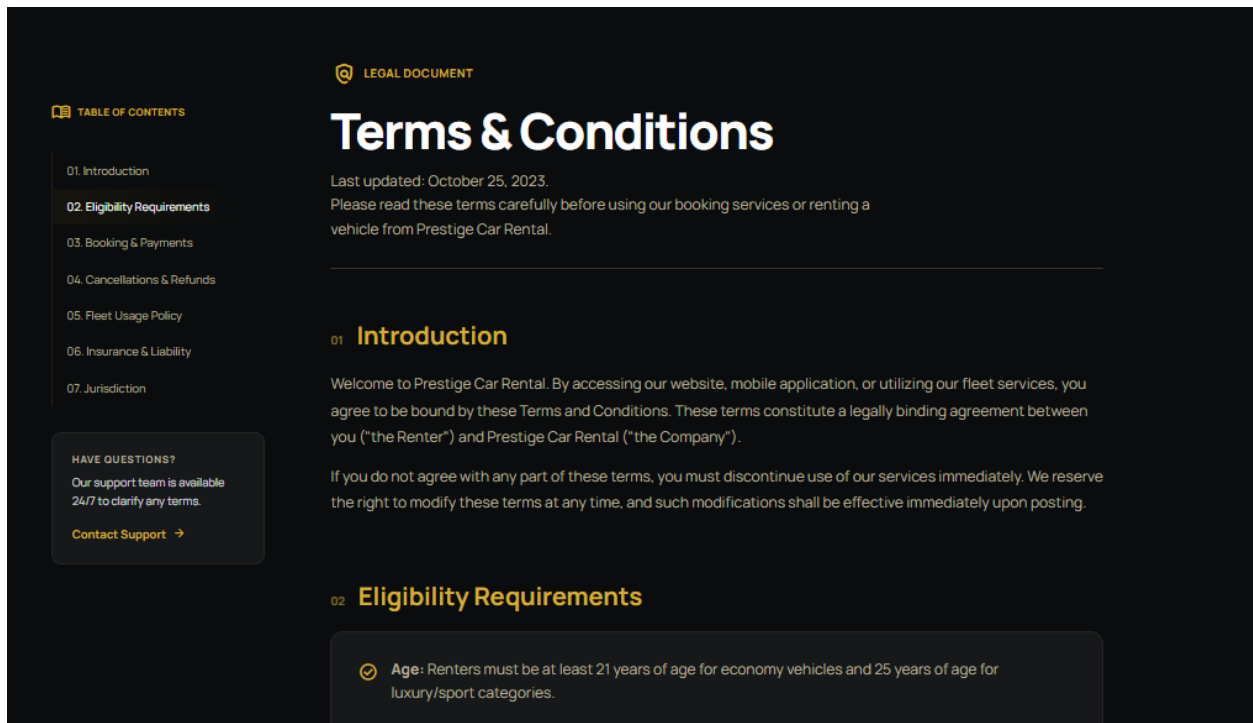


Figure 21 Terms & Conditions page

7.3.3.2 Browse Cars

Views/Browse/Index.cshtml + _CarGridItems.cshtml

Grid layout of cars with visuals and quick details.

Filters focus on real booking behavior: category, availability dates (though select days from valendar), and search by name/model.

Cards maintain consistent spacing and hover states to feel “shopping-like”.

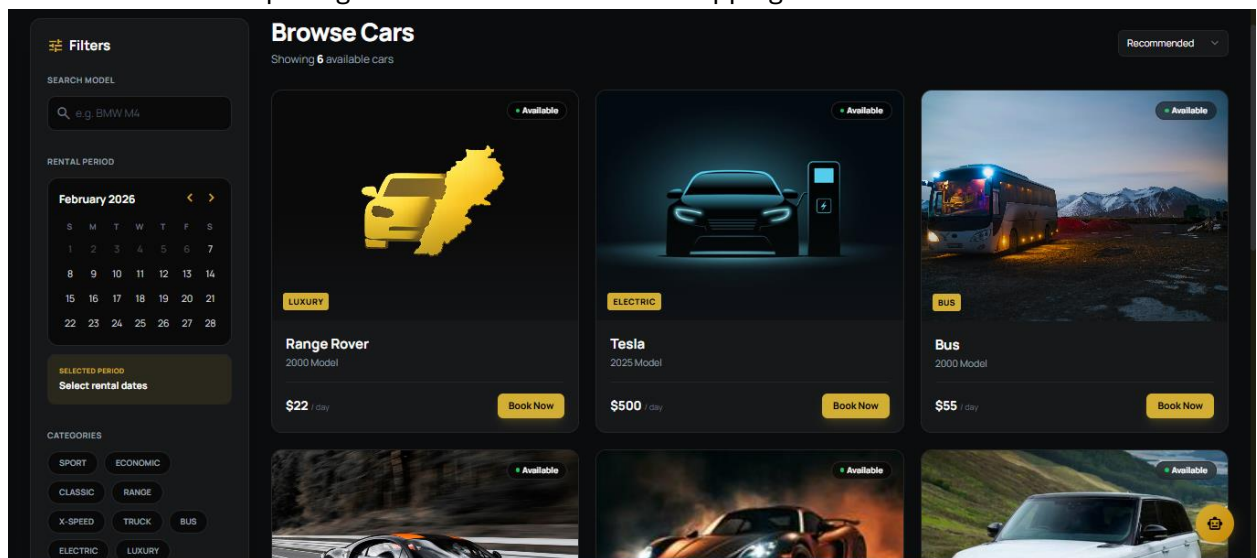


Figure 22 Browse Cars

7.3.3.3 Booking Creation

7.3.3.3.1 Views/Booking/Add.cshtml

Form-driven UI with validation.

Includes optional “Rent with Driver” section:

When enabled, it reveals pickup location inputs and map interactions.

Pricing is presented as a breakdown (subtotal + driver fee where applicable + discount/promo effects).

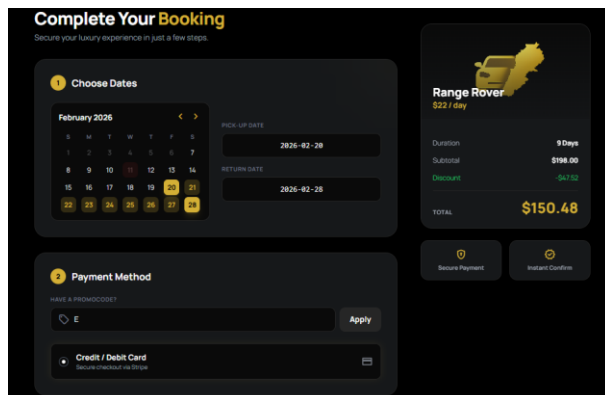


Figure 23 Make Booking

7.3.3.4 My Bookings + Details

7.3.3.4.1 Views/Bookings/MyBookings.cshtml + _CustomerBookingDetailsPartial.cshtml

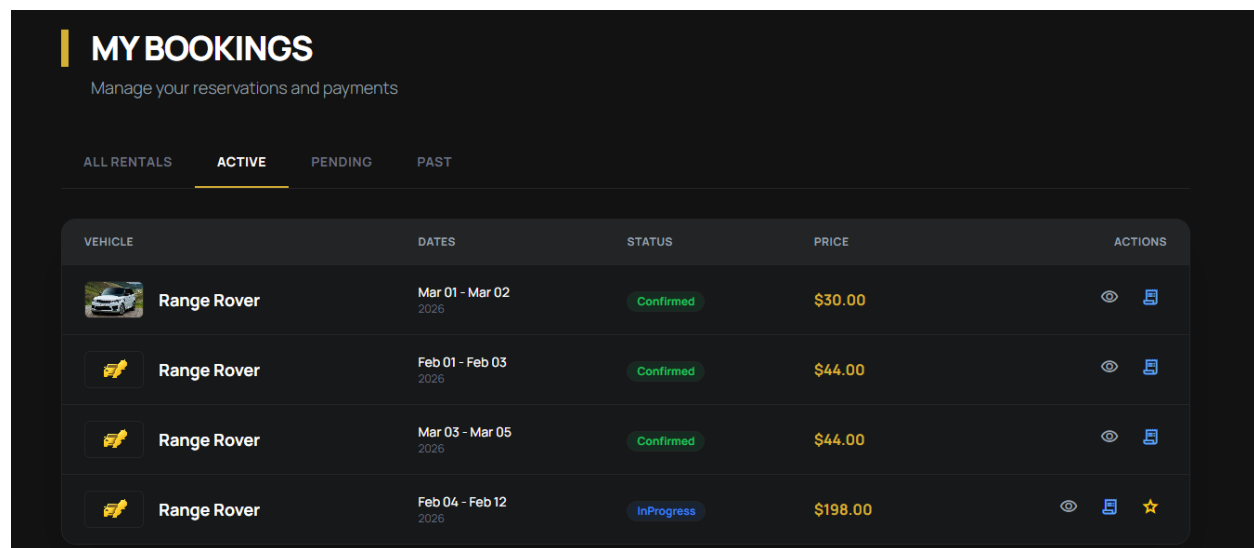
History and upcoming bookings in one place.

Each booking shows:

status progression (requested/confirmed/in-progress/waiting return/complete),

actions like view details, print receipt, and rate after completion.

If tracking is allowed/active, customers get direct access to tracking screens.

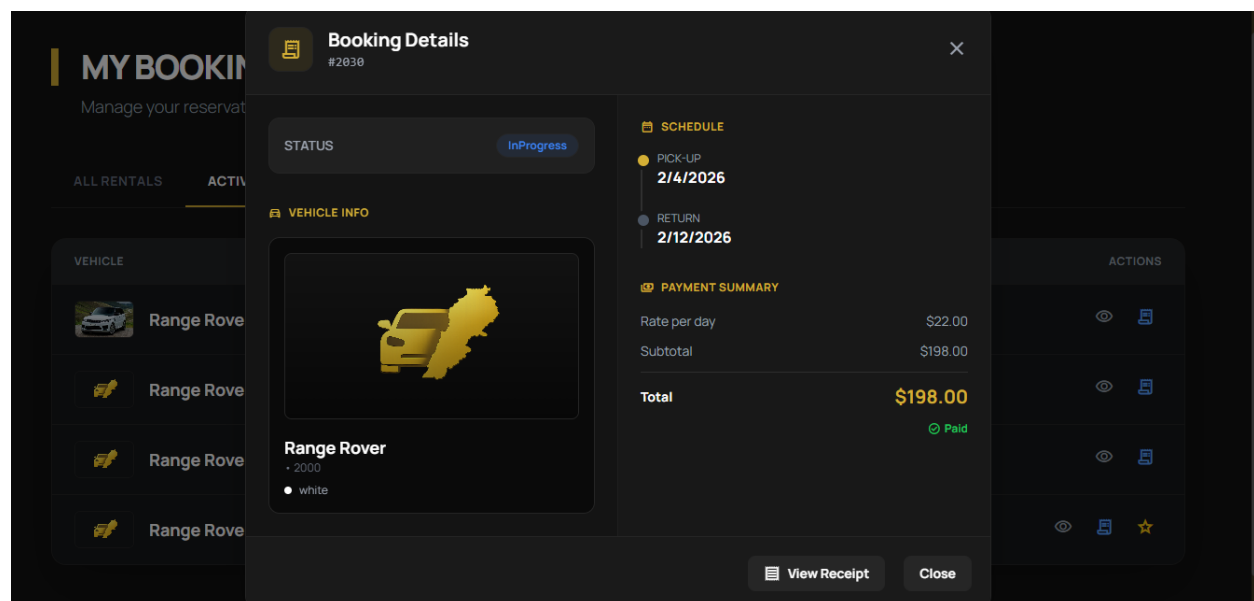


MY BOOKINGS
Manage your reservations and payments

ALL RENTALS **ACTIVE** PENDING PAST

VEHICLE	DATES	STATUS	PRICE	ACTIONS
 Range Rover	Mar 01 - Mar 02 2026	Confirmed	\$30.00	 
 Range Rover	Feb 01 - Feb 03 2026	Confirmed	\$44.00	 
 Range Rover	Mar 03 - Mar 05 2026	Confirmed	\$44.00	 
 Range Rover	Feb 04 - Feb 12 2026	InProgress	\$198.00	  

Figure 24 My Booking



Booking Details
#2830

STATUS
InProgress

VEHICLE INFO



Range Rover
• 2000
• white

SCHEDULE

PICK-UP
2/4/2026

RETURN
2/12/2026

PAYMENT SUMMARY

Rate per day \$22.00
Subtotal \$198.00
Total \$198.00
Paid

ACTIONS

 View Receipt  Close

Figure 25 View Details of booking (Customer side) - Partial in My bookings

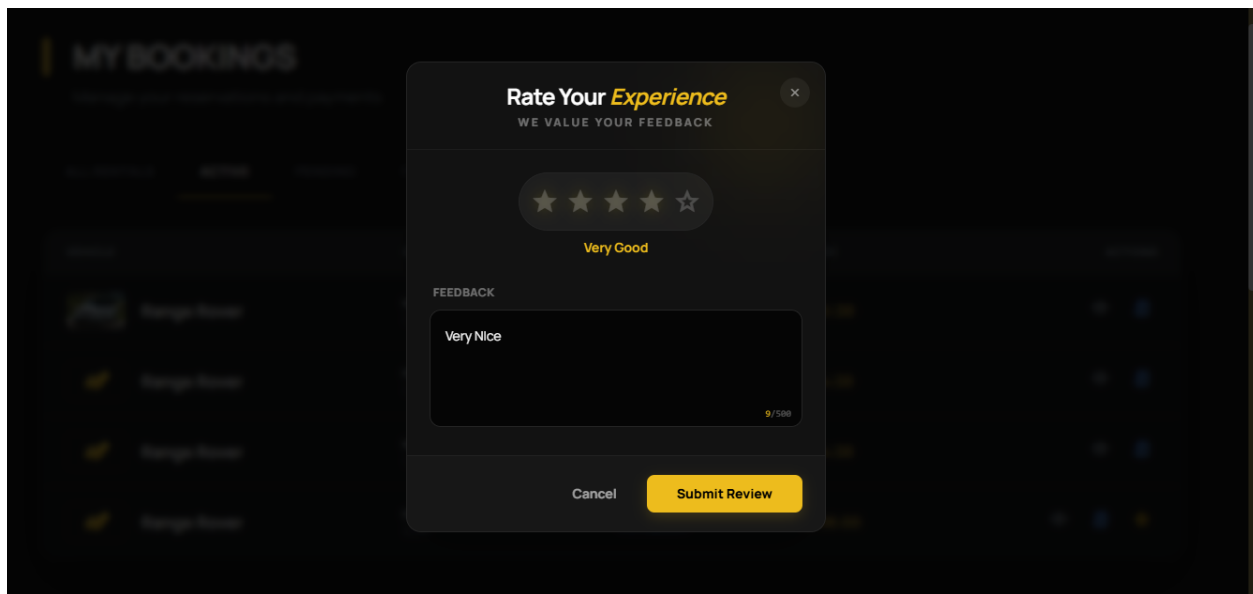


Figure 26 Rating - Customer side (partial in My bookings)

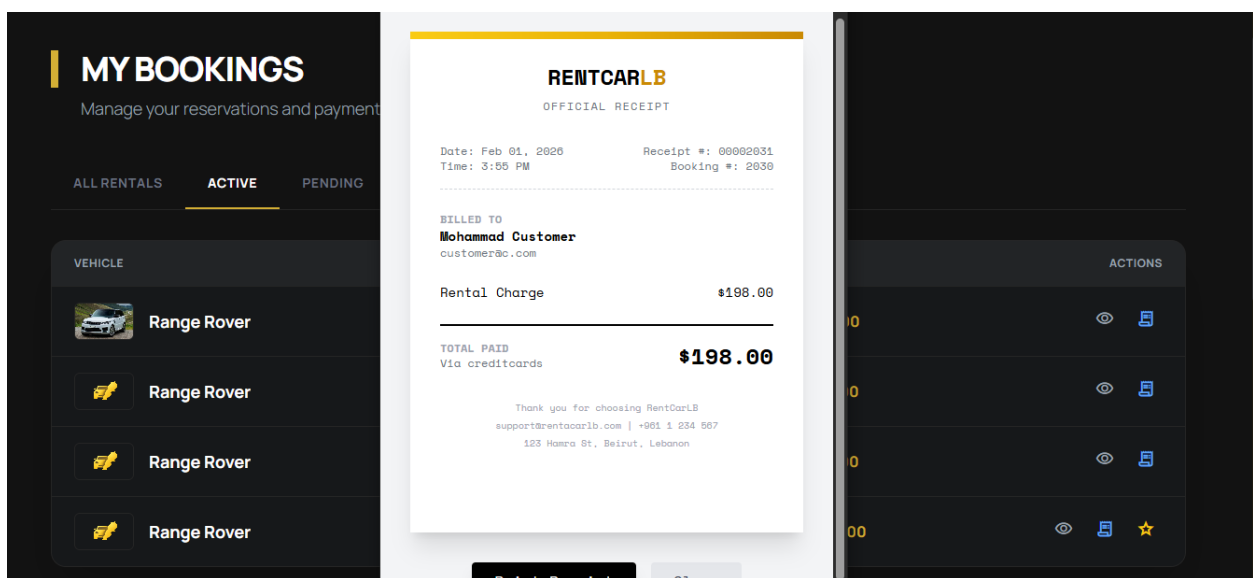


Figure 27 Booking Receipt print

7.3.3.5 Tracking

7.3.3.5.1 Views/Tracking/CustomerLive.cshtml

Map-first UI (dark mode map styling).

Clear separation between:

Customer location (blue glowing dot/pin when GPS offline),

Driver vehicle marker (car icon),

live movement updates to reduce uncertainty and missed pickups.

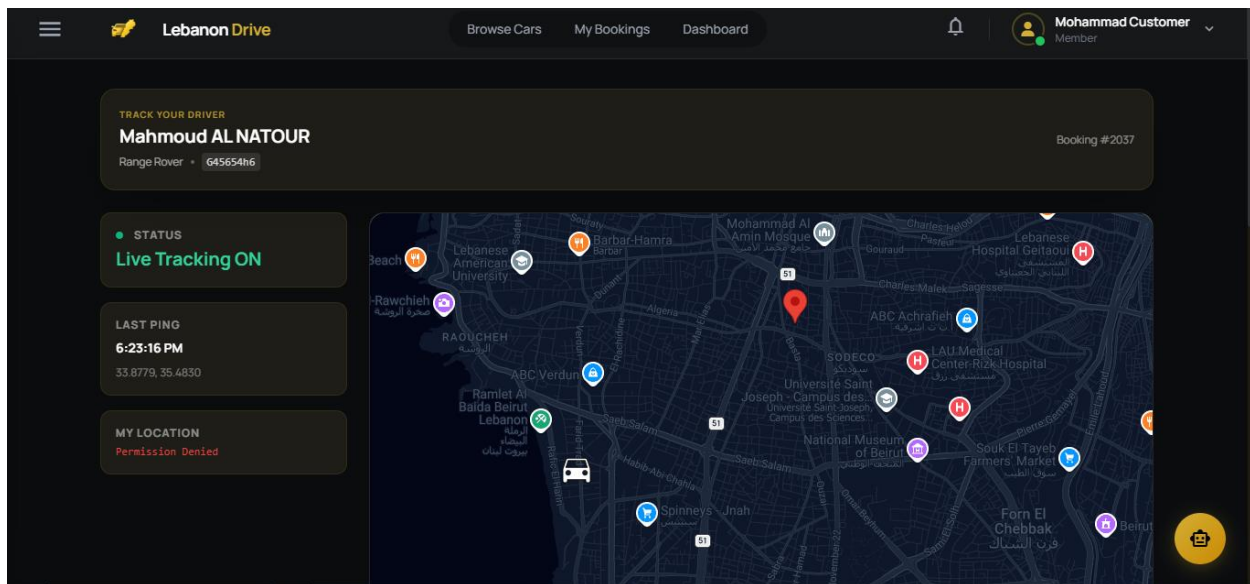


Figure 28 Tracking page (customer side)

7.3.3.6 Customer Support

7.3.3.6.1 Views/Support/Index.cshtml, Conversation.cshtml, _NewRequestModal.cshtml

Ticket creation modal: subject/category and optional booking link.

Conversation view resembles modern chat:

message bubbles aligned by sender,

timestamps,

status transitions (open → assigned/in-progress → resolved/closed).

UX goal: customers can solve issues without leaving the platform.

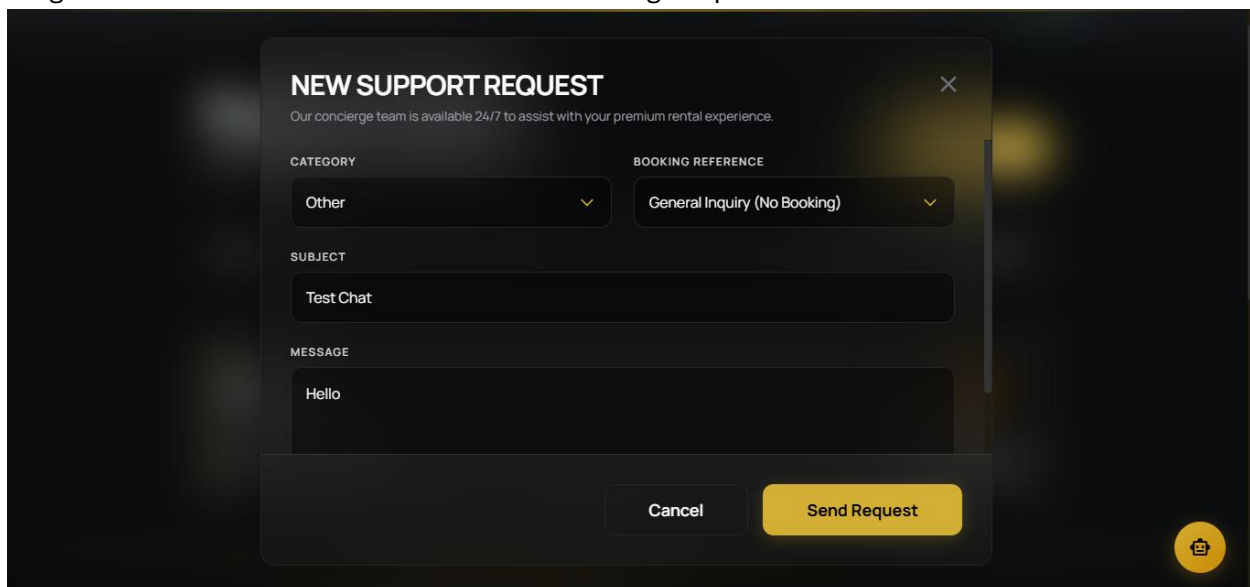


Figure 29 Create new ticket issue (Customer support)

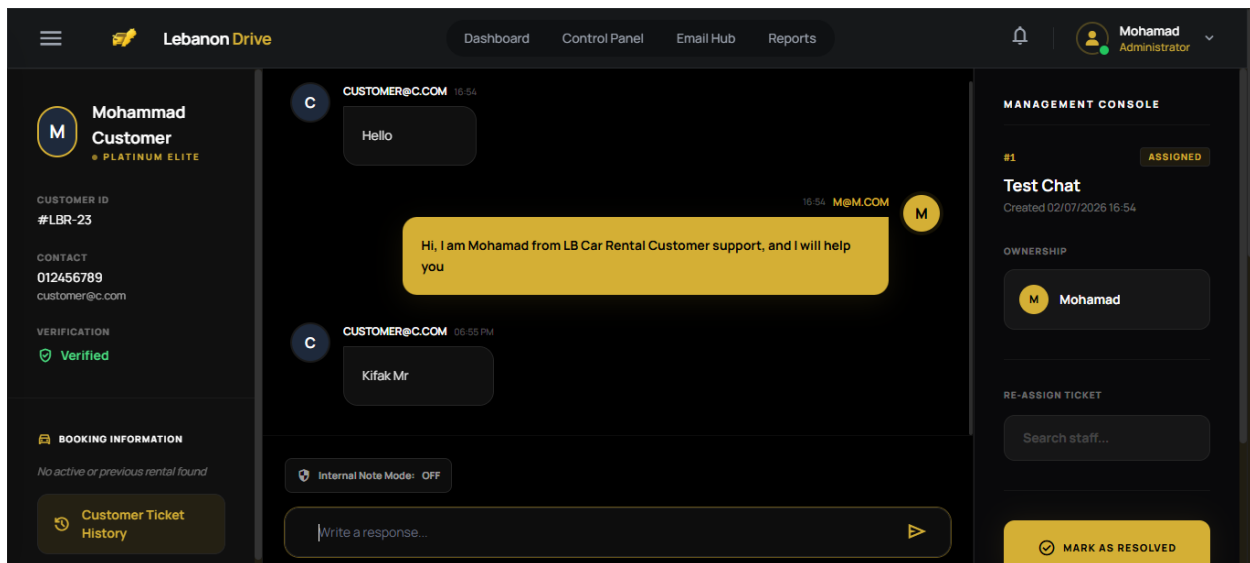


Figure 30 Chat page employee side

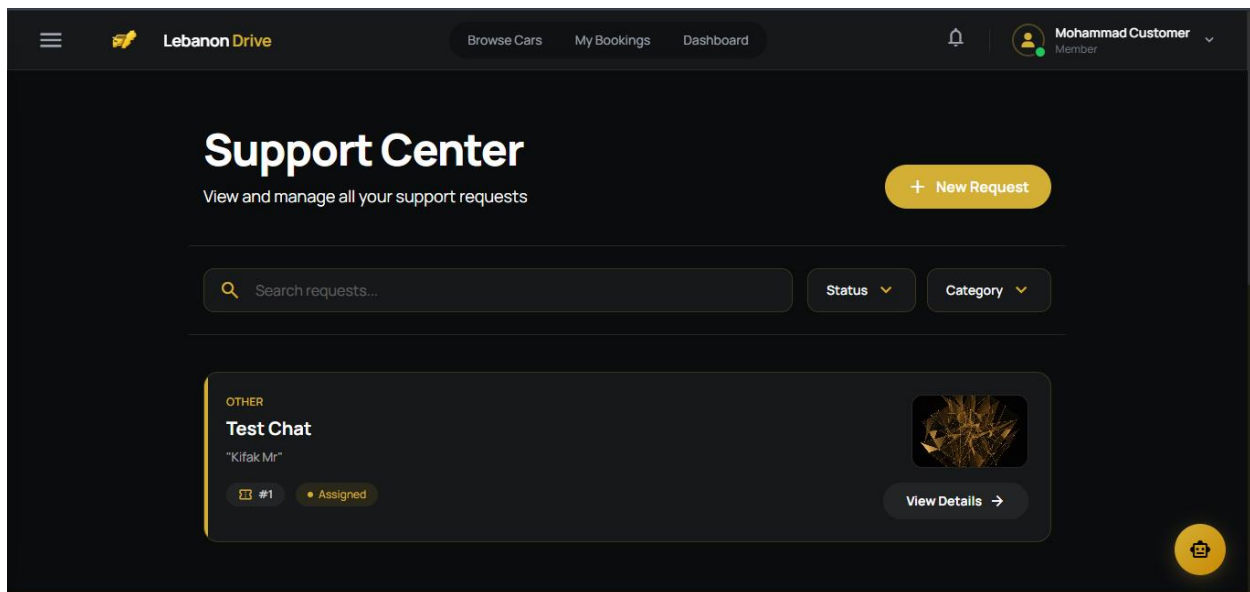


Figure 31List of tickets customer side

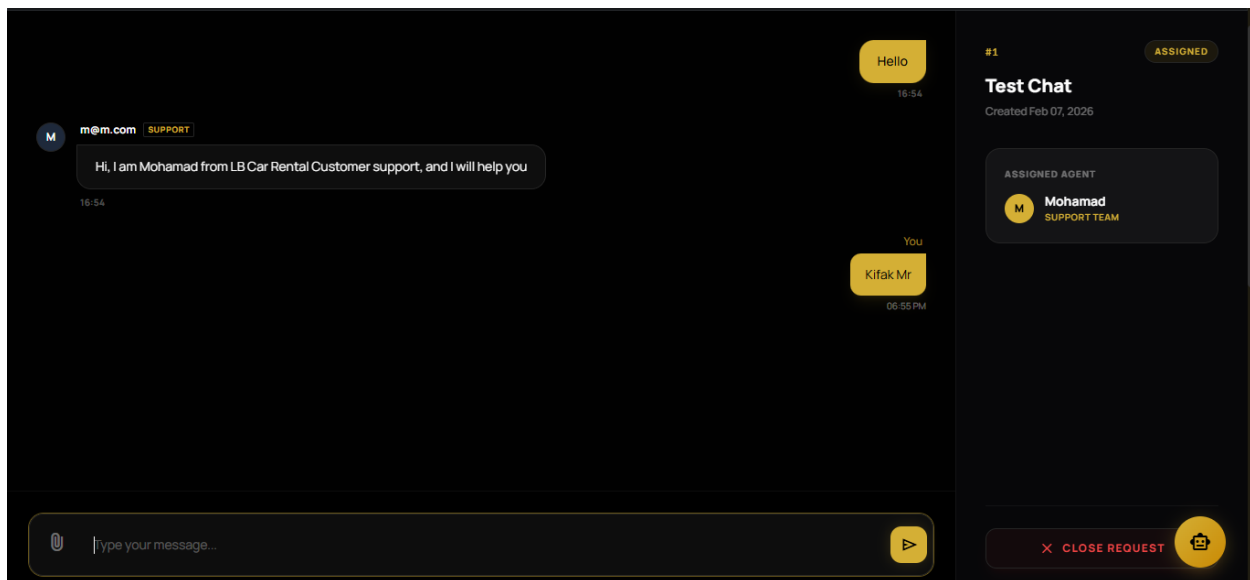


Figure 32 Chat page customer side

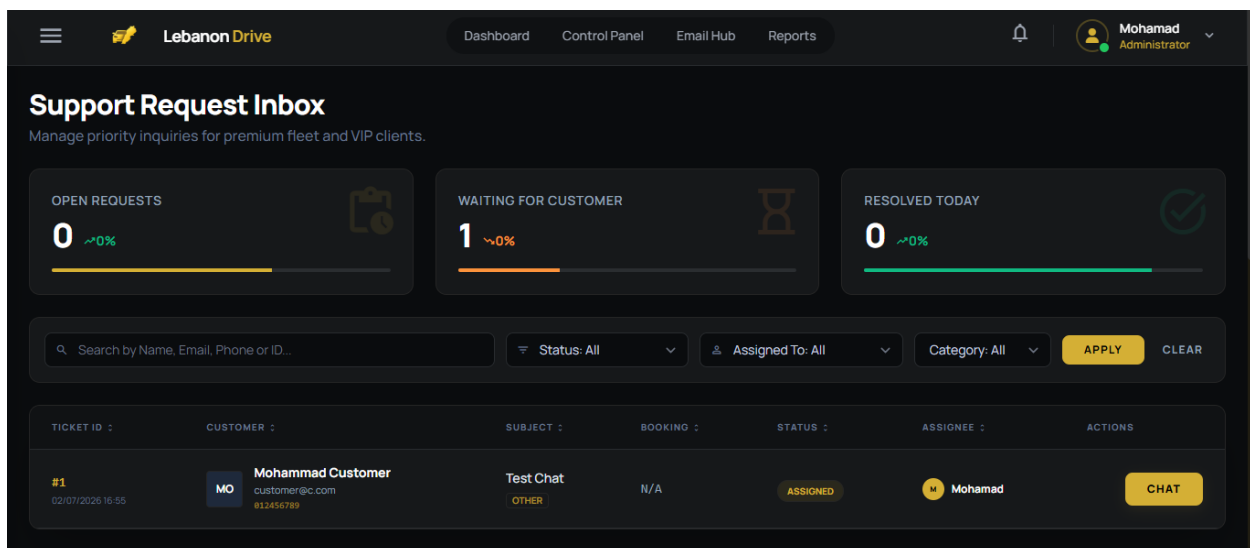


Figure 33 Customer Support Portal (employee access only)

7.3.3.7 AI Assistant (Customer Only)

7.3.3.7.1 Views/Shared/_AiChatWidget.cshtml + _AiEscalationModal.cshtml

Floating widget behavior (Chat-first).

“Escalate” path triggers a modal that converts context into a support request when automation isn’t enough.

Keeps the UI lightweight by hiding it for employee/admin/driver roles.

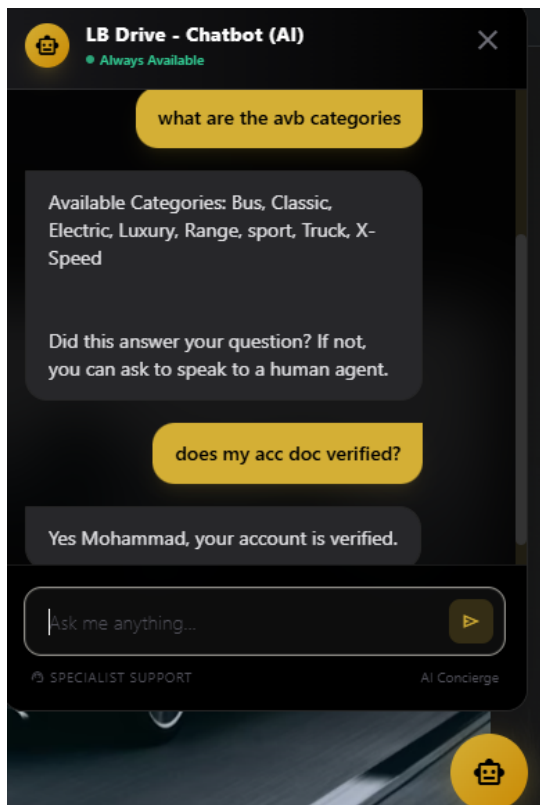


Figure 34 AI Chatbot

7.3.3.8 Employee Portal (Operational UI)

Employees see the same modern UI shell, but their navigation and page access are restricted:

They can **view cars/categories/payment methods** and work with bookings/customers/support, but can't perform admin-only operations (blacklist, role changes, employee management, provider-level email settings).

Key pages they interact with:

Bookings management (customer walk-in booking creation + status updates like pickup/return).

Customer management (verification review workflows).

Support inbox (handle assigned conversations).

Email hub features (send email, drafts, templates depending on permissions), and logs visibility.

Dashboard of employee

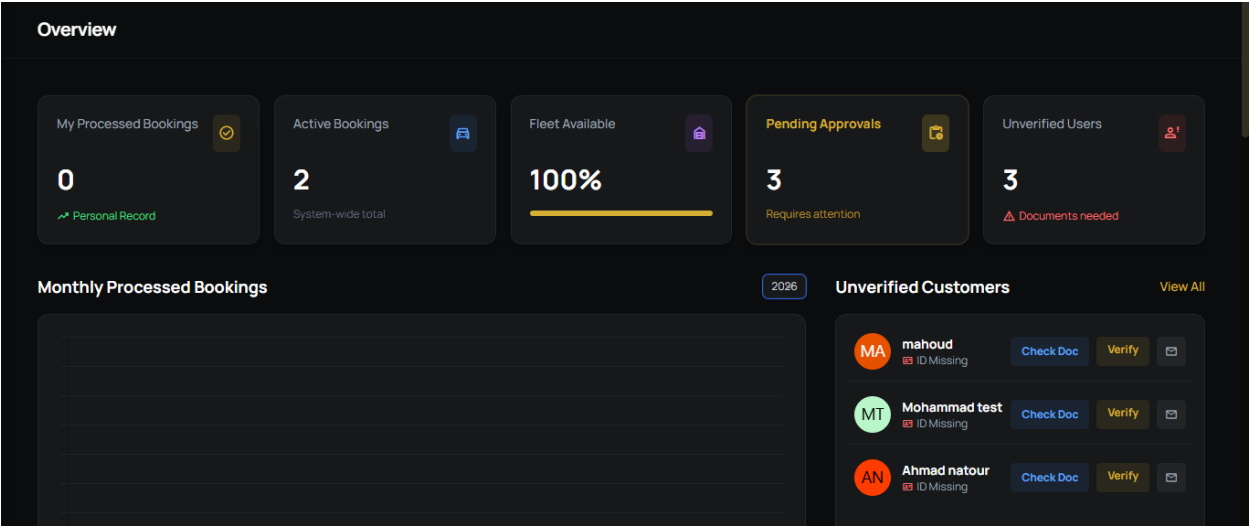


Figure 35 Employee Dashboard section 1

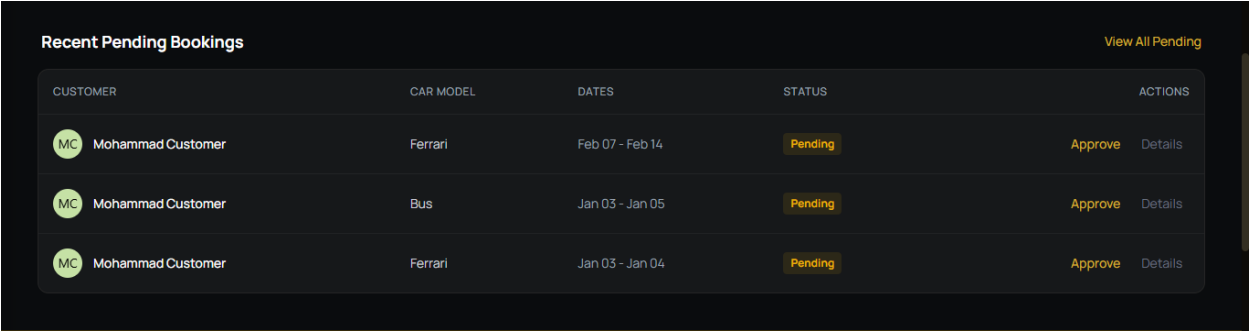


Figure 36 Employee Dashboard section 2

7.3.3.9 Admin Control Panel UI (Management Hub)

7.3.3.9.1 Control Panel Home (Card-Based Hub)

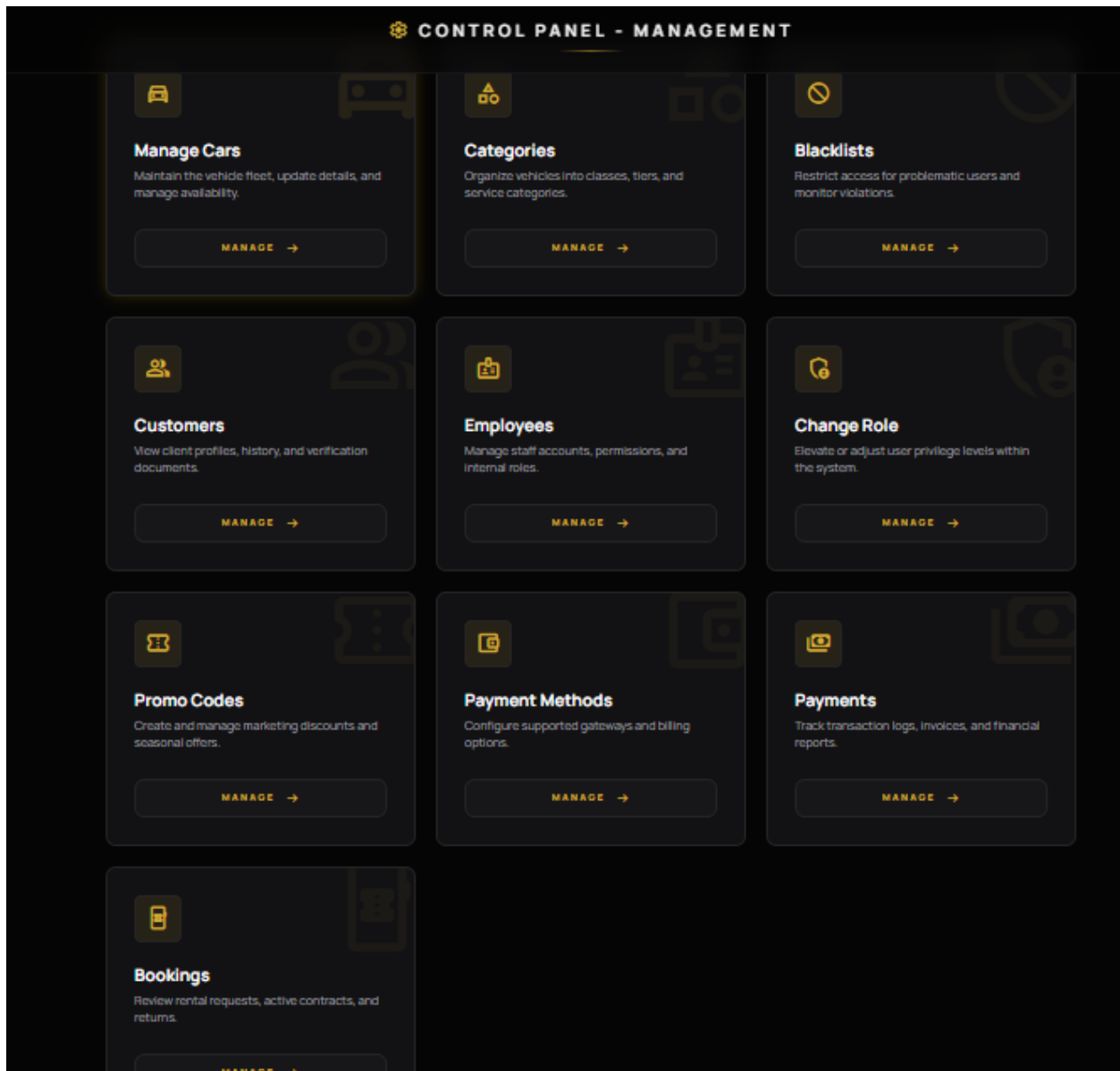


Figure 37 Control - Management panel

7.3.3.9.1.1 Views/ControlPanel/Index.cshtml

A grid of “module cards” such as:

Manage Cars, Categories, Customers, Employees,
Blacklist,
Promo Codes,
Payment Methods and Payments,
Bookings,

Change Role (admin-only).

Each card contains:

icon + short description,

a “Manage” CTA (consistent across modules).

7.3.3.9.1.2 CRUD Pages

Views/ControlPanel/ modules (Car, Category, Booking, Customer, Employee, PaymentMethod, Promocode, Ratings, Blacklist, etc.)

Table-first layout for admin productivity:

searchable lists,

actions per row,

modals or separate pages for create/edit,

delete confirmation modal (`_DeleteConfirmationModal.cshtml`).

7.3.3.10 Reports + Audit Log

Views/Reports/index.cshtml + AuditLog.cshtml

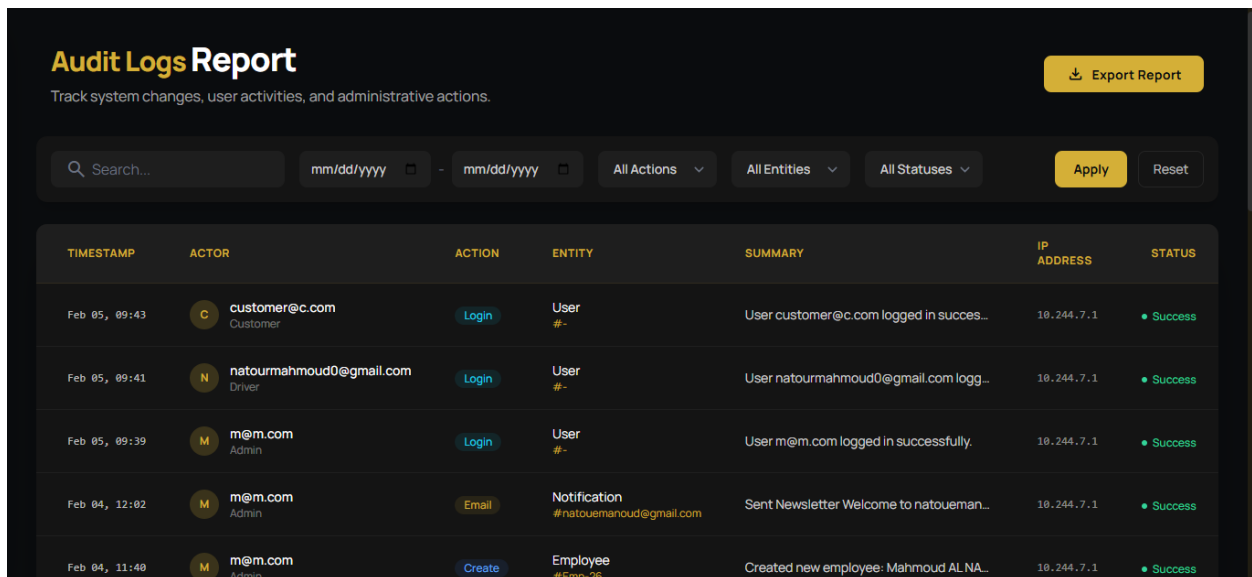
Report screens are designed as dashboards:

filterable date ranges,

aggregated metrics blocks,

tables/charts depending on report type.

Audit logs use a dense, admin-friendly layout with readable event summaries.



TIMESTAMP	ACTOR	ACTION	ENTITY	SUMMARY	IP ADDRESS	STATUS
Feb 05, 09:43	 customer@c.com Customer	Login	User #-	User customer@c.com logged in succes...	10.244.7.1	Success
Feb 05, 09:41	 natourmahmoud0@gmail.com Driver	Login	User #-	User natourmahmoud0@gmail.com logg...	10.244.7.1	Success
Feb 05, 09:39	 m@m.com Admin	Login	User #-	User m@m.com logged in successfully.	10.244.7.1	Success
Feb 04, 12:02	 m@m.com Admin	Email	Notification #natouemanoud@gmail.com	Sent Newsletter Welcome to natoueman...	10.244.7.1	Success
Feb 04, 11:40	 m@m.com Admin	Create	Employee #Emp-26	Created new employee: Mahmoud AL NA...	10.244.7.1	Success

Figure 38 Audit Logs

7.3.3.11 Email Services Hub (Mini-Gmail UI Concept)

7.3.3.11.1 Hub Overview

Views/EmailServices/EmailServicesHub.cshtml

Matches the screenshot:

Top KPI strip (sent today, delivery rate, active reminders, pending errors).

Cards for core modules:

Send Email,

Distribution Lists,

Email Configuration Center (admin-only),

Templates,

Logs/Outbox,

Notification Log.

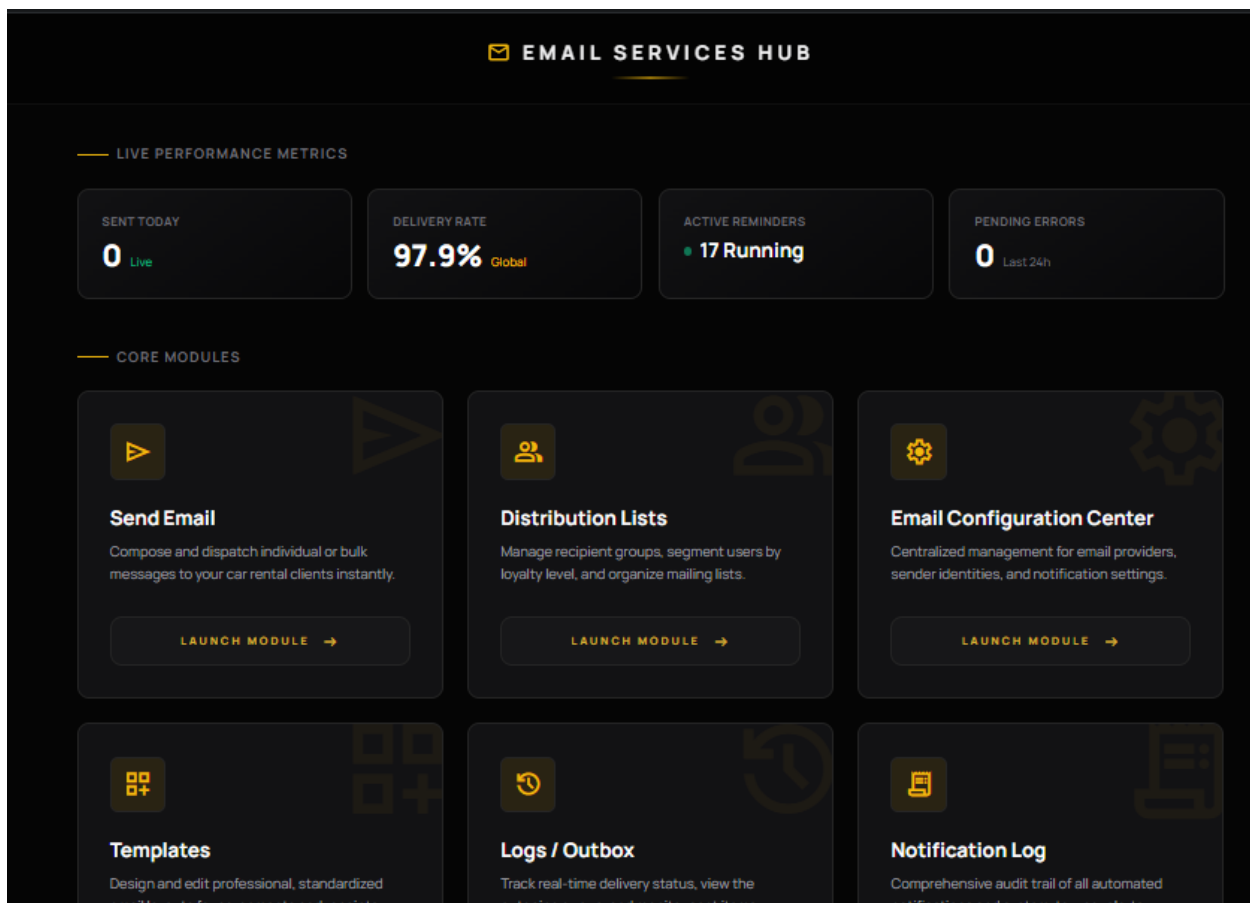


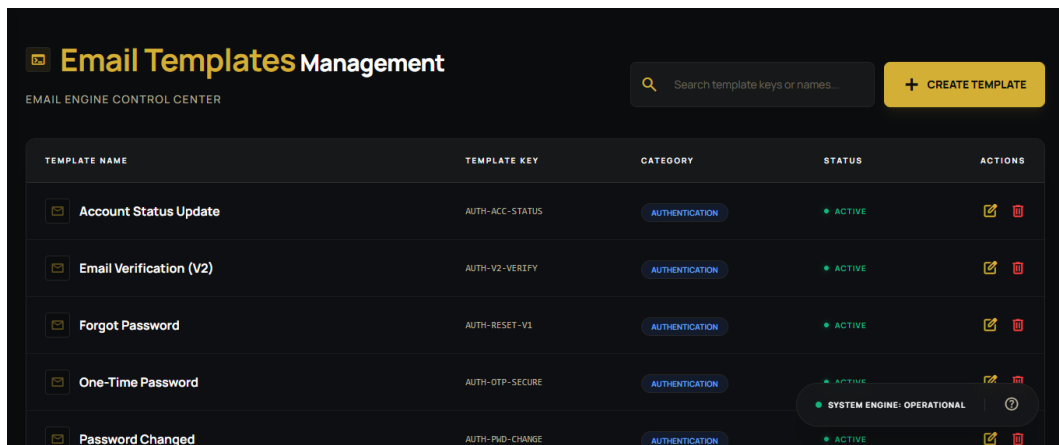
Figure 39 Email Service Hub (Management)

7.3.3.11.2 Templates + Live Preview

Views/EmailServices/Templates/

Template list + editor pages.

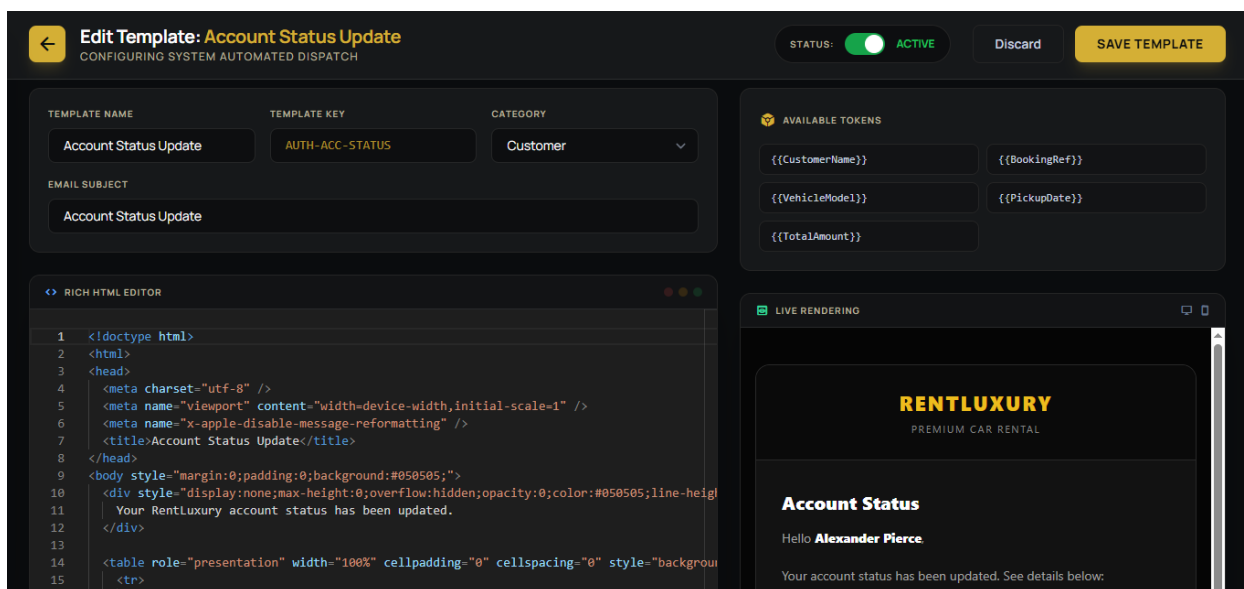
HTML editing is paired with a **preview panel** (“live compiler” concept) so staff can validate formatting before linking templates to features.



The screenshot shows the 'Email Templates Management' interface. At the top, there's a search bar and a '+ CREATE TEMPLATE' button. Below is a table with columns: TEMPLATE NAME, TEMPLATE KEY, CATEGORY, STATUS, and ACTIONS. The table lists five templates: Account Status Update, Email Verification (V2), Forgot Password, One-Time Password, and Password Changed. Each template has a status of 'ACTIVE' and a category of 'AUTHENTICATION'. The 'One-Time Password' row also shows a 'SYSTEM ENGINE: OPERATIONAL' status.

TEMPLATE NAME	TEMPLATE KEY	CATEGORY	STATUS	ACTIONS
Account Status Update	AUTH-ACC-STATUS	AUTHENTICATION	ACTIVE	[Edit] [Delete]
Email Verification (V2)	AUTH-V2-VERIFY	AUTHENTICATION	ACTIVE	[Edit] [Delete]
Forgot Password	AUTH-RESET-V1	AUTHENTICATION	ACTIVE	[Edit] [Delete]
One-Time Password	AUTH-OTP-SECURE	AUTHENTICATION	ACTIVE	[Edit] [Delete]
Password Changed	AUTH-PWD-CHANGE	AUTHENTICATION	ACTIVE	[Edit] [Delete]

Figure 40 Email templates lists



The screenshot shows the 'Edit Template: Account Status Update' interface. It includes a status toggle set to 'ACTIVE', a 'Discard' button, and a 'SAVE TEMPLATE' button. The form contains fields for TEMPLATE NAME (Account Status Update), TEMPLATE KEY (AUTH-ACC-STATUS), CATEGORY (Customer), and EMAIL SUBJECT (Account Status Update). There is a 'RICH HTML EDITOR' showing HTML code and a 'LIVE RENDERING' preview. The preview shows a 'RENTLUXURY' account status update for 'Alexander Pierce'.

Figure 41 Create template

7.3.3.11.3 Outbox / Logs and Notification Log (Admin Area)

Areas/Admin/Views/Outbox/ and **Areas/Admin/Views/NotificationLog/**

Log tables for traceability.

Detail modals for inspecting payloads, recipients, errors, and retry behavior.

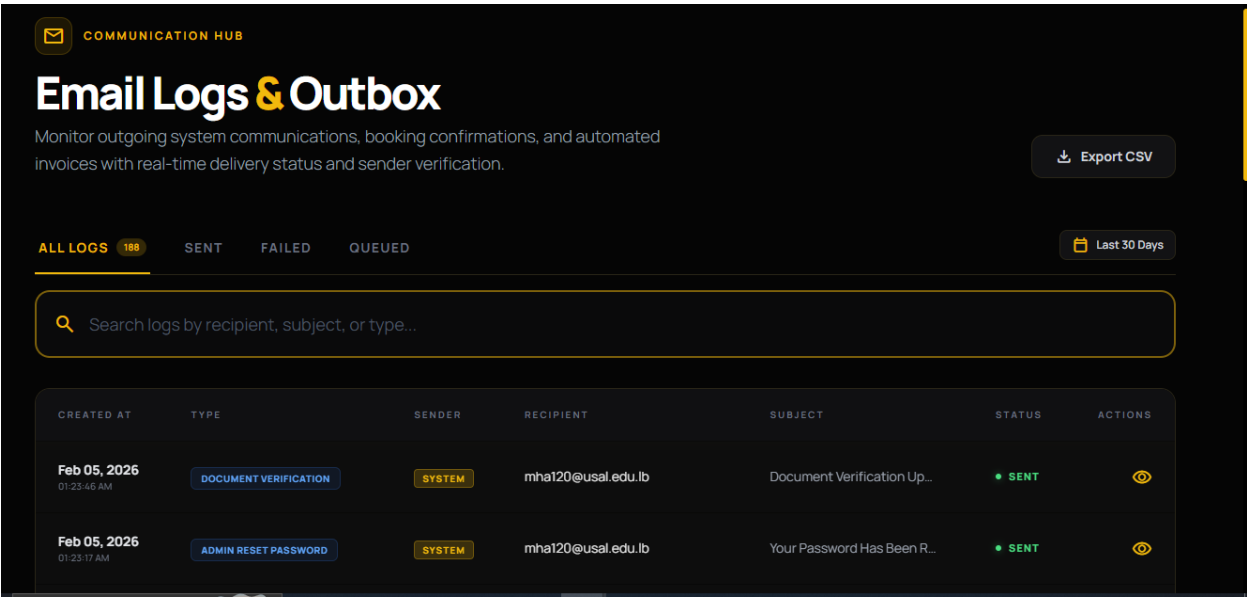


Figure 42 Email logs

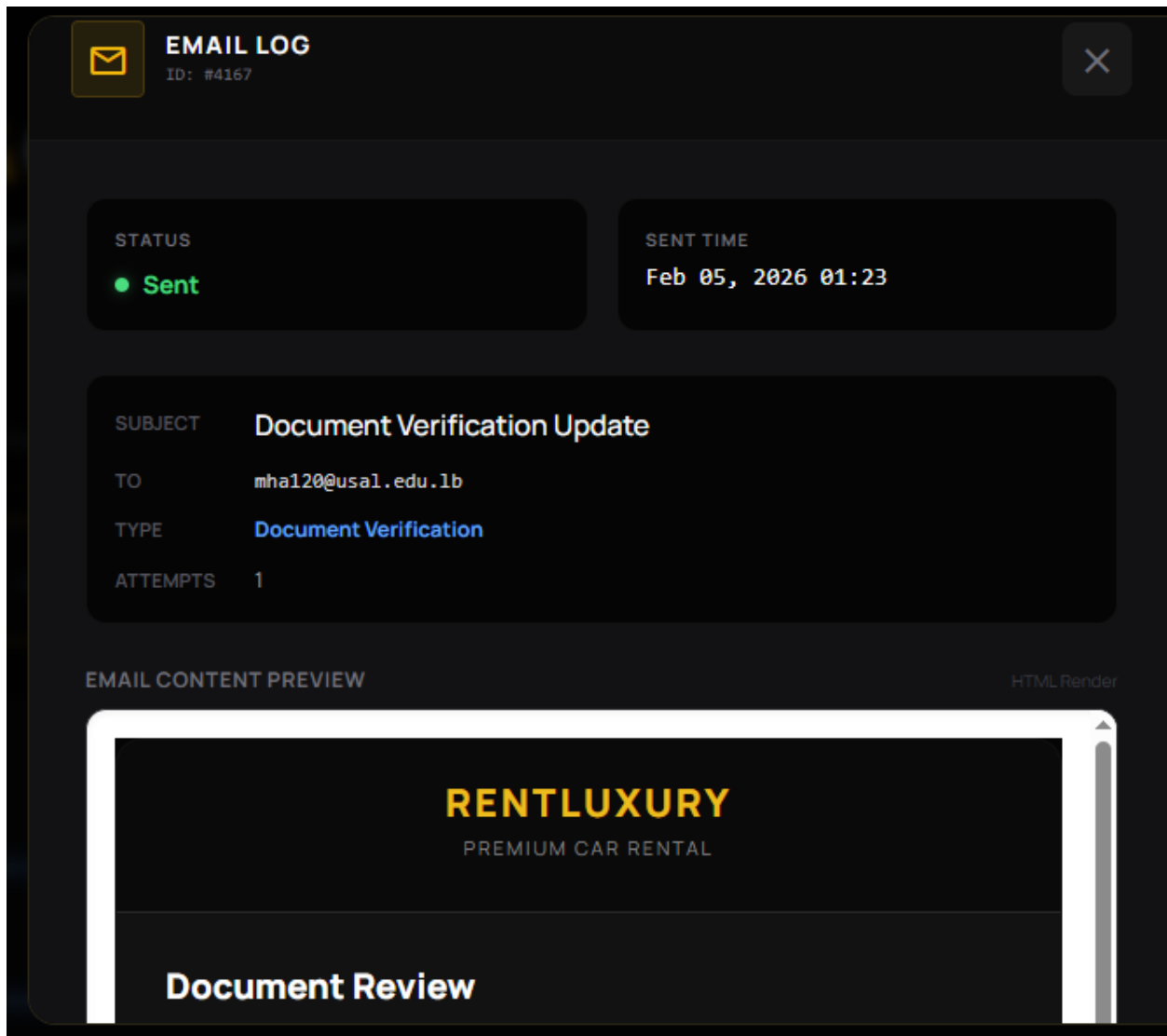


Figure 43 Email details

7.3.3.12 Driver Portal UI

7.3.3.12.1 Views/DriverPortal/

Dashboard: today's missions, quick status buttons (e.g., go online), live map preview.

Schedule: calendar-like planning view for upcoming trips.

7.3.3.12.2 BookingDetails + TripDetails partials:

structured information panels so the driver sees pickup info, booking reference, and trip state.

The UI is map-centered to support real operations, not just administration.

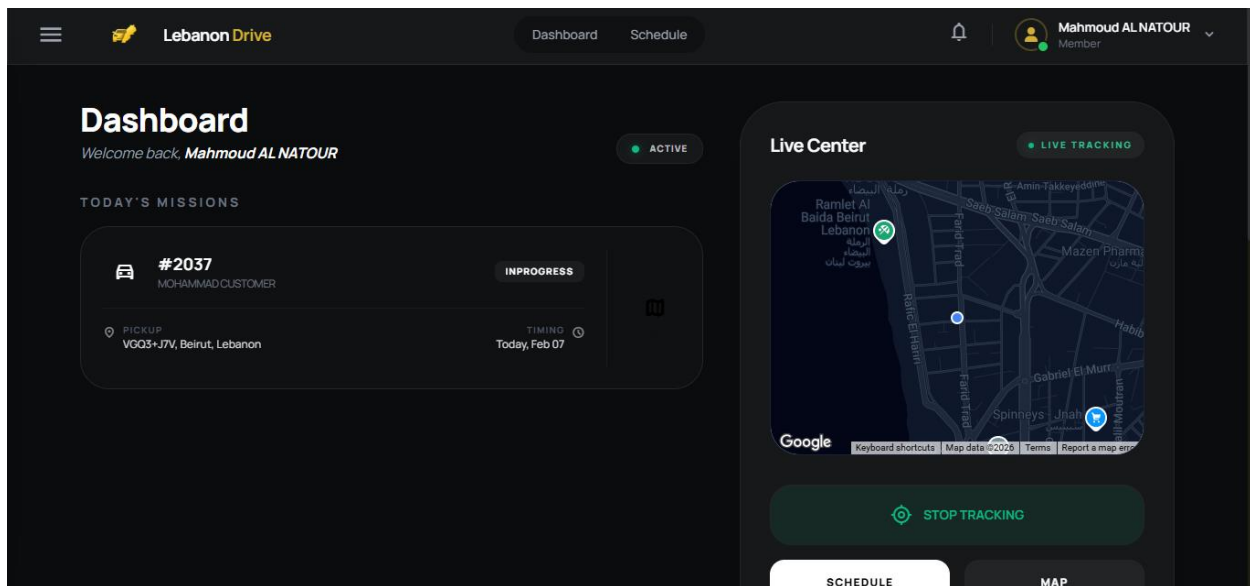


Figure 44 Driver Dashboard'

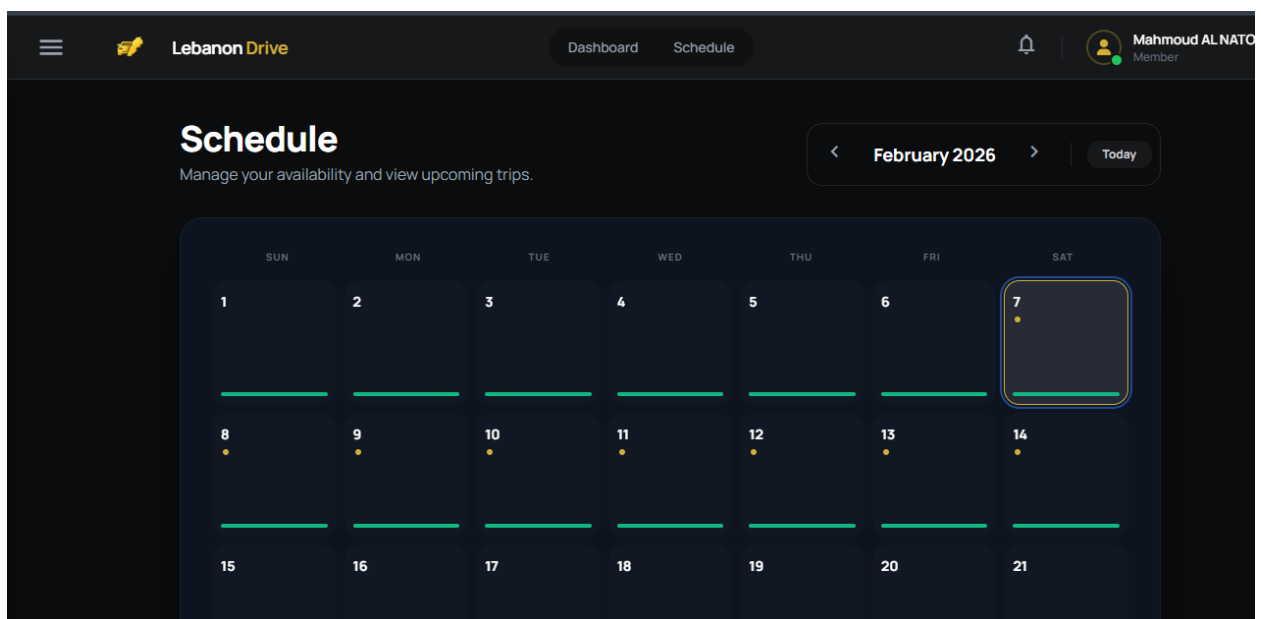


Figure 45 Driver Schedule

7.3.3.13 Identity (Authentication & Security Pages UI)

Areas/Identity/Pages/Account/

Covers full authentication lifecycle:

Login, Register, Logout
 Forgot/reset password flows
 Email confirmation/resend

Lockout / Access denied

Areas/Identity/Pages/Account/Manage/

Account management is treated as a “profile dashboard”:

Profile basics (Index)

Email change / password change

2FA flows:

enable authenticator,

recovery codes,

disable/reset authenticator,

login with recovery/2FA pages

VerifyIdentity.cshtml: identity verification upload (driving license + government ID), aligned with the customer booking eligibility workflow.

The screenshot displays the 'ACCOUNT SETTINGS' page for a user named Mahmoud AL NATOUR. The page is divided into several sections. At the top, there's a navigation bar with 'Lebanon Drive' and links to 'Dashboard' and 'Schedule'. Below this, the 'ACCOUNT SETTINGS' title is prominent, followed by a tabbed interface. The 'Profile' tab is selected, showing 'PERSONAL INFORMATION' with a subtext 'Update your public profile and contact details.' The form contains four main input fields: 'DISPLAY NAME' (filled with 'Mohammad Ayoubi'), 'PHONE NUMBER' (filled with '+96178989360'), 'HOME ADDRESS' (filled with 'Beirut Tariq Jdide'), and 'EMAIL ADDRESS' (filled with 'natourmahmoud0@gmail.com'). A 'LOCKED' status with a padlock icon is visible next to the email field.

Figure 46 Account Settings

7.3.3.14 Admin Dashboard



Figure 47Admin Dashboard

7.3.4 UI/UX Principles Applied

Consistency: Same layout + typography + color language across all modules.

Role clarity: UI presents only what a role needs (customer vs employee vs admin vs driver).

Action-driven dashboards: “cards → manage” pattern speeds navigation for staff.

Operational feedback: popups/toasts, badges, logs, and status chips reduce ambiguity.

Mobile responsiveness: sidebar overlay + flexible grids (cards collapse naturally).

Trust & compliance cues: verification prompts, audit logs, security pages, and controlled access to sensitive modules.

8 Development

This section describes how the designed modules were implemented in the actual system, including the database schema, UI, integrations, and the major workflows across all roles (Customer, Employee, Admin/Manager, Driver, Support).

8.1 Project Structure and Implementation Approach

The implementation follows a **4-project N-Layer solution**:

RentACar.Core: domain entities + enums + core rules (e.g., Booking, Trip, Support/Ai conversations) + IRepository

RentACar.Application: managers (Business logic) + some outside services + DTOs.

RentACar.Infrastructure: EF Core DbContext, repositories, external providers (Email, Stripe, AI), background services.

RentACar.Web: MVC Controllers + Views + Areas (Admin/Identity), UI components, SignalR/live interactions.

This separation kept controllers thin and ensured that rules like “customer must be verified before booking” and “avoid booking conflicts” remain in business logic, not in UI code.

8.2 Database Schema and EF Core Integration

The database is implemented in **SQL Server** and managed through **Entity Framework Core migrations**. The schema combines:

ASP.NET Identity tables (AspNetUsers, AspNetRoles, ...) for authentication/authorization

Business tables for rental operations (Bookings, Cars, Customers, Payments, Promocodes, ...)

Advanced subsystems tables for AI chat, support chat, driver tracking, and email automation.

Key implemented relationships from the live schema include:

Customers.aspNetUserId → AspNetUsers.Id, Employees.aspNetUserId → AspNetUsers.Id,
Drivers.aspNetUserId → AspNetUsers.Id

Bookings.customerID → Customers.userID, Bookings.carID → Cars.CarID, optional
Bookings.EmployeebookerID → Employees.employeeID

Support subsystem: SupportConversations and SupportMessages with assignment to employees

Driver subsystem: Trips, DriverAvailabilities, and DriverLocationPings

Email subsystem: templates, drafts, lists, provider settings, feature configs, logs, notifications, and background run records

8.3 Authentication, Authorization, and Security Features

Authentication is built using **ASP.NET Core Identity** with role-based authorization (Customer/Employee/Admin/Driver). Sensitive operations are protected at both:

UI level (navigation hides restricted modules),

Controller/action level (authorization filters / role checks).

Security-related features implemented:

Customer verification workflow: customers must upload driving license + government ID and wait employee approval before booking.

Multi-factor authentication (MFA):

email OTP,
authenticator (TOTP),
recovery/backup codes.

Account recovery (forgot/reset password flows via Identity pages).

Audit logging: security-sensitive actions are written to AuditLogs with actor metadata (role, device/ip when available) and old/new values when relevant.

Blacklist enforcement: blocked accounts are stored in BlackList and restricted from sensitive workflows (especially booking and platform access depending on business rule enforcement).

8.4 Customer Portal Development

The customer-facing portal is implemented in MVC using controllers and views under:

BrowseController + Views/Browse/* for car browsing and filtering

BookingController + Views/Booking/* for booking creation

BookingsController + Views/Bookings/* for booking history/details/receipt/rating

TrackingController + Views/Tracking/* for live tracking access

SupportController + Views/Support/* for tickets + chat

Shared UI widgets in Views/Shared/* (layout, navbar, sidebar, popups, AI widget)

Implemented customer flows:

Register/Login → optional MFA

Upload verification documents → wait verification

Browse cars → filter by category/date/name → view car details

Create booking:

choose dates, promocode,

optional “rent with driver” → pickup location + map section,

pay online (Stripe card only) for online bookings

My Bookings:

view history and details,

print receipt,

rate after completion

Notifications: booking status changes and reminders are delivered by the email subsystem.

8.5 Employee Operations Development

Employee operations are implemented through:

CustomerController (customer management + verification)

booking workflows (BookingController/BookingsController)

support inbox/chat (Support module access)

limited email access (send/drafts/templates/lists/logs) without provider configuration pages.

Employee permissions (as implemented in the system design):

Can: create and verify customers, create in-person bookings, update booking status, handle support tickets/chat, reset customer passwords, view reference data (cars/categories/payment methods), manage promo codes, use email hub operational features.

Cannot: manage employees, change roles, manage blacklist, access full email configuration center, disable cars and categories.

8.6 Admin Control Panel Development

The admin management portal is centered around ControlPanelController and related controllers:

CarController, CategoryController, EmployeeController, PaymentMethodController, PaymentController, PromocodeController, RatingsManagementController, BlacklistController, ReportsController.

Admin has full CRUD across the platform, including:

cars/categories/customer/employee management,

payment methods and payment management,

bookings management and reporting,

blacklist management,

audit log visibility,

full email hub including configuration center.

The UI follows a “management hub” approach (cards → module entry), consistent with the screenshots you provided.

8.7 Booking Lifecycle and Conflict Prevention

Booking logic is enforced in business layer services/managers and includes:

Availability validation to prevent overlapping bookings for the same car and date range.

Status progression across the lifecycle:

Pending (while/waiting payment done)

Confirmed (after Stripe success / after staff creation),

In Progress (car delivered / contract signed),

Waiting to Return (auto/operational stage when overdue),

Completed/Closed (after return).

Rejected (Payment Failed)

Canceled (done by employee side)

Optional driver flow:

hasDriver = 1 + driver fee per day stored,

pickup coordinates and label stored in booking,

trip created/linked in Trips and updated by driver status.

8.8 Stripe Payment Integration

Online booking uses **Stripe card payments only** via StripeController and Views/Stripe/*.

Implementation includes:

creating Stripe checkout/session,

storing provider identifiers (paymentProviderSessionId, paymentProviderPaymentIntentId) in Payments,

updating booking/payment status upon confirmation,

logging events into the email/notification subsystem for confirmation and receipts.

In-person payments (cash/card) are recorded by staff and stored in Payments without invoking Stripe.

8.9 Driver Module and Live Tracking Development

Driver workflows are implemented via:

DriverPortalController + Views/DriverPortal/*

TrackingController + Views/Tracking/*

Driver features:

schedule of assigned trips,

update trip status (mission lifecycle stored in Trips.tripStatus),

location ping sending stored in DriverLocationPings (speed/heading/accuracy supported),

live map tracking view for customer/staff:

customer shown as blue glowing dot (Google Maps style),

driver shown as moving car marker,

dark-mode map styling for consistent UI.

8.10 Customer Support (Ticket + Live Chat) Development

Support is implemented through:

SupportController and Views/Support/*

Admin area support inbox under Areas/Admin/Views/SupportInbox/* for management visibility.

Main implemented logic:

customer creates a ticket (SupportConversations, status Open),

employee claims/opens → ticket gets assigned (AssignedEmployeeId) and moves to In Progress,

messages stored in SupportMessages (supports internal notes and optional attachment URL),

employee resolves → customer can close,

all conversations remain viewable after closure for accountability.

8.11 AI Assistant Development (Customer Only)

AI assistant is exposed only for customer accounts through:

UI widget in shared views,

API endpoints under Controllers/Api/*,

persistence via AiConversations and AiMessages.

Design implemented:

the assistant loads **context** (logged-in user + cars/categories + booking constraints/availability window) and answers common questions.

escalation path: if AI can't resolve, it triggers the support workflow (creates a ticket and passes chat context).

8.12 Email Services Hub Development (Mini-Gmail + Automation)

The email subsystem is implemented via controllers:

EmailServicesController, SendEmailController, TemplatesController, DistributionListsController, EmailConfigurationCenterController
and admin monitoring under:

Areas/Admin/Views/Outbox/*, Areas/Admin/Views/NotificationLog/*.

Core capabilities implemented:

composing and sending emails,

saving drafts (EmailDrafts),

template CRUD (EmailTemplates) including HTML editing + preview,

recipient management through distribution lists and rules (DistributionLists, members, rules),

feature-to-template mapping (EmailFeatureConfigs) to link events (OTP, booking status changed, reminders, etc.) to templates,

provider settings and sender identities (EmailProviderSettings, SenderIdentities) — admin-only,

logging and observability:

sent/outbox (EmailLogs),

system notification log (NotificationLogs),

background run tracking (ServiceRunRecords, ServiceRunItems),

runtime control (NotificationSettings).

Background reminders/notifications run as hosted/background services and are controlled from settings, ensuring email automation is measurable and auditable.

8.13 UI Implementation (Layouts, Components, Identity Pages)

The web UI is implemented with:

shared layout, navbar, sidebar, footer, and popup notifications under Views/Shared/*,

role dashboards under Views/Dashboard/*,

admin pages grouped by management hubs (control panel + email hub),

Identity pages under Areas/Identity/Pages/Account/* including:

login/register,

forgot/reset password,

2FA pages (authenticator, recovery codes),

verification upload page (VerifyIdentity.cshtml).

The interface applies consistent dark mode styling and gold accent for actions and module cards, and it uses responsive navigation (collapsible sidebar + role-based menu items).

8.14 Testing, Deployment, and Deployment Considerations

Testing and validation focused on:

booking conflict scenarios,

verified vs unverified customer behavior,

Stripe success/failure flows,

ticket assignment and chat persistence,

driver location ping persistence and map display consistency,

email automation execution and log correctness,

role restrictions (employee vs admin).

Deployment considerations included:

secure environment configuration (API keys for Stripe, Email provider, Gemini, Maps),

database migration readiness,

background services reliability and logging,

scalability-friendly separation of external provider logic in Infrastructure.

9 Challenges

This project faced technical, operational, and resource constraints typical of building a real-world system without a production budget or enterprise partnerships. The main challenges and how they were handled are summarized below.

9.1 External API and Partner Constraints (Flight Ticketing Removed)

At the start, the project scope included **flight ticketing integration** alongside car rental. During requirements validation, it became clear that flight ticketing APIs are generally restricted to **registered travel agencies or licensed companies**, and providers require official business verification and legal agreements. Since no company could grant API access under student/project conditions, the flight module was removed to avoid delivering a partially functional feature.

Lesson learned: scope must match what can be legally and practically integrated, not just what is technically possible.

9.2 Payment Gateway Limitations (Production Access Not Possible)

A major challenge was enabling real online payments. Production-grade payment gateways typically require:

- a bank account under a registered business,

- legal documents and compliance checks.

The team explored local options (including **Whish Money**) but could not obtain an integration path suitable for this project. As a result, the system uses **Stripe in sandbox mode (student plan)** for online card payments, while in-person rentals support cash/card recorded by staff.

How it was addressed: the payment workflow was implemented end-to-end (checkout session, provider IDs stored, booking status updates, and receipt generation) while keeping the gateway in a sandbox environment.

Lesson learned: implement payments in a way that can later switch to production keys with minimal refactoring.

9.3 Budget and Hosting Instability (Multiple Server Moves)

Hosting required continuous adaptation. The system was initially hosted using an **Azure trial**, but once the free trial expired the project needed a replacement quickly. Due to budget limits, the deployment was moved across **multiple servers (three hosting changes)** to keep the application online and stable.

How it was addressed:

- configuration was moved to environment variables,

deployment was simplified and repeated consistently,

the team focused on portability (connection strings, API keys, and provider settings separated from code).

Lesson learned: deployment portability matters; avoid hardcoding environment-specific configurations.

9.4 AI Assistant Provider Selection, Cost Constraints, and Logic responses

The AI assistant was not straightforward to finalize because:

many AI providers are expensive for continuous usage,

some require paid tiers for stable access,

API terms and quotas vary widely.

After evaluating multiple options, the implementation stabilized on **Gemini Flash 2.0 (free tier)** to meet budget constraints.

A second challenge was making the AI responses “system-aware” instead of generic. Early results were often unrelated to business rules.

How it was addressed: the assistant was redesigned to include **context injection** with each message (logged-in user info, cars, categories, booking constraints/availability window, and relevant system policies). Escalation to human support was added when AI confidence is low.

Lesson learned: AI quality depends more on structured context + guardrails than on the model alone.

9.5 Complexity Growth (Many Modules in One Platform)

The project expanded into a full operational platform: bookings, verification, payments, driver workflow, live tracking, support chat, AI assistant, email hub, background reminders, audit logs, and admin management. The risk was featuring overlap and inconsistent rules across modules.

How it was addressed:

strict role-based access control (customer/employee/admin/driver),

modular design (control panel hub, email hub, support module),

centralized business rules in the service/manager layer.

Lesson learned: as scope grows, structure becomes more important than adding features quickly.

9.6 Booking Conflicts and Data Consistency

Preventing double-booking required careful handling of date overlap logic and status transitions (confirmed → in progress → waiting return → completed). This became more complex with:

optional driver scheduling,

pickup location metadata,

reminders and automation.

How it was addressed: conflicts were validated server-side using date-overlap checks, and booking lifecycle rules were enforced in business logic rather than UI.

Lesson learned: availability rules must be enforced centrally and tested with edge cases.

9.7 Background Services Reliability (Reminders + Notification Logging)

The email subsystem includes background jobs (payment reminders, pickup reminders, return reminders, promo expiry alerts, status notifications, etc.). The challenge was making scheduled automation reliable and traceable.

How it was addressed:

execution state stored in NotificationSettings,

each run recorded in ServiceRunRecords and item-level outcomes in ServiceRunItems,

all sends tracked in EmailLogs and NotificationLogs.

Lesson learned: automation must be observable; logs and run records are essential.

9.8 Time Constraints and Team Coordination

A practical challenge was time. Most contributors worked **at night**, which created:

slower feedback loops,

limited overlap for pair debugging,

higher risk of merge conflicts.

How it was addressed:

splitting responsibilities by module,

using commits and incremental integration,

stabilizing core workflows first (booking + verification + payments) before advanced modules.

Lesson learned: coordination strategy matters as much as coding when schedules don't align.

10Future Work

The current system delivers a functional, secure prototype that covers end-to-end rental operations. However, several enhancements can significantly increase its commercial readiness and research value, especially since the long-term goal is to **productize and sell the platform**.

10.1.1 Payment Gateway Reintegration (Real Production Payments)

Currently, online payments are implemented using a sandbox environment. A key future step is to integrate production payment options:

Enable **Stripe production** with real business verification (and/or local gateways once business agreements exist).

Add more payment flexibility (wallets, installment providers if available locally).

Introduce fraud detection rules and stronger reconciliation for financial records.

10.1.2 Advanced AI Assistant (Beyond API Calls)

The current AI assistant works through an external API and is guided using context injection. For a commercial product, AI can be improved in multiple directions:

Replace the “API-only chatbot” with a more robust architecture:

retrieval-based knowledge base (RAG) using internal policies, FAQs, booking rules, and system data,

richer tool calling (availability checks, booking lookup, support status) with strict permission control.

Improve grounding and accuracy by using structured prompts, guardrails, and evaluation tests.

Add multilingual support (Arabic/English/French) since this is important in Lebanon.

Extend AI to employee/admin workflows (e.g., assisting with reports, anomaly detection, or customer verification assistance), while keeping privacy boundaries.

10.1.3 Mobile Application Development

A high-impact improvement is building a mobile application (or cross-platform app):

Customer mobile app for browsing, booking, tracking, support chat, and document upload.

Driver mobile app optimized for:

trip status updates,

location pinging in background,

navigation and pickup workflow.

This would improve usability and adoption, especially for real-world driver operations.

10.1.4 Expanded Roles and Privilege Control (Accounting, HR, Fine-Grained Permissions)

To sell the system as a business platform, additional enterprise roles are valuable:

Accounting role: invoice review, payment reconciliation, revenue reports, tax/export features.

HR role: staff management, attendance integration (optional), driver records, performance tracking.

Privilege panel: implement granular permissions (not only role-based) so each company can customize access (e.g., employee can edit promos but cannot send bulk email; support staff can chat but cannot access customer documents).

10.1.5 Loyalty Program and Customer Retention Features

A loyalty layer can help rental companies retain customers:

Points or tier program based on completed bookings.

Discounts for frequent renters, referral codes, birthday promotions.

Personalized offers triggered by booking history, integrated into the Email Services Hub campaigns.

10.1.6 Data Analytics and Business Intelligence Improvements

Current reporting can be expanded into deeper analytics:

fleet utilization optimization (idle rate, top-performing categories, seasonal demand),

driver performance metrics (pickup delays, completion time),

support metrics (time to first response, resolution time),

automated anomaly alerts (suspicious booking patterns, repeated payment failures).

10.1.7 Stronger Compliance and Document Verification Enhancements

Future improvements can strengthen verification and legal compliance:

integrate OCR to extract driver license/ID data (optional),

add manual review checklist workflows for employees,

support digital signatures on contracts,

introduce retention policies for sensitive images (automatic deletion after a defined period).

11 Conclusion

This project delivered a complete web-based Rent-a-Car Management System designed to replace the paper- and spreadsheet-driven workflows still common in many mid-sized Lebanese rental companies. The implemented platform successfully digitizes the full operational lifecycle: customer onboarding with identity verification, online and in-person booking management with conflict prevention, payment handling (Stripe-based online card payments and staff-recorded in-person payments), contract and receipt generation, driver-enabled rentals with trip tracking, and centralized reporting and monitoring.

A major outcome is that the system does not stop at “online booking.” Instead, it provides an end-to-end operational solution that covers all key roles—customer, employee, administrator/manager, driver, and customer support—each with dedicated dashboards, role-based navigation, and access controls. The project also introduced features that are uncommon in small local systems but important for real deployment, including multi-factor authentication, audit logging, customer blacklisting, structured support ticketing with live chat, and a full email operations hub with templates, drafts, distribution lists, event-to-template mapping, and background reminder services.

The project demonstrates practical value by reducing human error and administrative overhead, improving booking reliability through server-side validation, and improving customer experience through online accessibility, automated notifications, and real-time operational visibility. From a software engineering perspective, the use of a layered architecture, Identity-based security, and modular subsystems (payments, email, AI, support, tracking) shows how a scalable platform can be built even under strict constraints such as limited budget, restricted third-party APIs, and hosting instability.

The implications extend beyond the immediate use case. The system provides a foundation for future applied work in operational automation, service quality improvements using AI assistance, and enterprise-ready extensions such as production payment gateways, mobile applications, expanded roles (accounting/HR), and loyalty programs. Overall, the project contributes a realistic, extensible prototype that demonstrates how digital transformation can be achieved for local rental businesses, and it offers a strong base for further research and commercialization.

12 Bibliography

Technical documentation and standards:

[1] Microsoft, “ASP.NET documentation,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/>. [Accessed: Feb. 7, 2026].

[2] Microsoft, “Get started with ASP.NET Core MVC,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/start-mvc?view=aspnetcore-10.0>. [Accessed: Feb. 7, 2026].

[3] Microsoft, “Entity Framework Core (EF Core) documentation,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/ef/core/>. [Accessed: Feb. 7, 2026].

[4] Microsoft, “SQL Server technical documentation,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver17>. [Accessed: Feb. 7, 2026].

[5] Bootstrap Team, “Bootstrap documentation (v5.3),” Bootstrap. [Online]. Available: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>. [Accessed: Feb. 7, 2026].

- [6] Stripe, “Checkout Sessions API,” Stripe Docs. [Online]. Available: <https://docs.stripe.com/api/checkout/sessions>. [Accessed: Feb. 7, 2026].
- [7] Stripe, “Compare Checkout Sessions and Payment Intents APIs,” Stripe Docs. [Online]. Available: <https://docs.stripe.com/payments/checkout-sessions-and-payment-intents-comparison>. [Accessed: Feb. 7, 2026].
- [8] Google, “Maps JavaScript API overview,” Google Developers. [Online]. Available: <https://developers.google.com/maps/documentation/javascript/overview>. [Accessed: Feb. 7, 2026].
- [9] Microsoft, “Multi-factor authentication in ASP.NET Core,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/mfa?view=aspnetcore-10.0>. [Accessed: Feb. 7, 2026].
- [10] Google, “Gemini API documentation,” Google AI for Developers. [Online]. Available: <https://ai.google.dev/gemini-api/docs>. [Accessed: Feb. 7, 2026].
- [11] Google Cloud, “Gemini 2.0 Flash,” Vertex AI documentation. [Online]. Available: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash>. [Accessed: Feb. 7, 2026].
- [12] Mailjet, “Email API reference,” Mailjet Developer Docs. [Online]. Available: <https://dev.mailjet.com/email/reference/>. [Accessed: Feb. 7, 2026].

Software engineering and architecture:

- [13] R. C. Martin, Clean Architecture: A Craftsman’s Guide to Software Structure and Design. Upper Saddle River, NJ, USA: Prentice Hall, 2017.
- [14] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA, USA: Addison-Wesley, 2003.
- [15] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

Security and compliance references:

- [16] OWASP Foundation, “OWASP Top Ten,” OWASP. [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: Feb. 7, 2026].
- [17] PCI Security Standards Council, “PCI DSS v4.0.1 (Document Library),” PCI SSC. [Online]. Available: https://www.pcisecuritystandards.org/document_library/. [Accessed: Feb. 7, 2026].